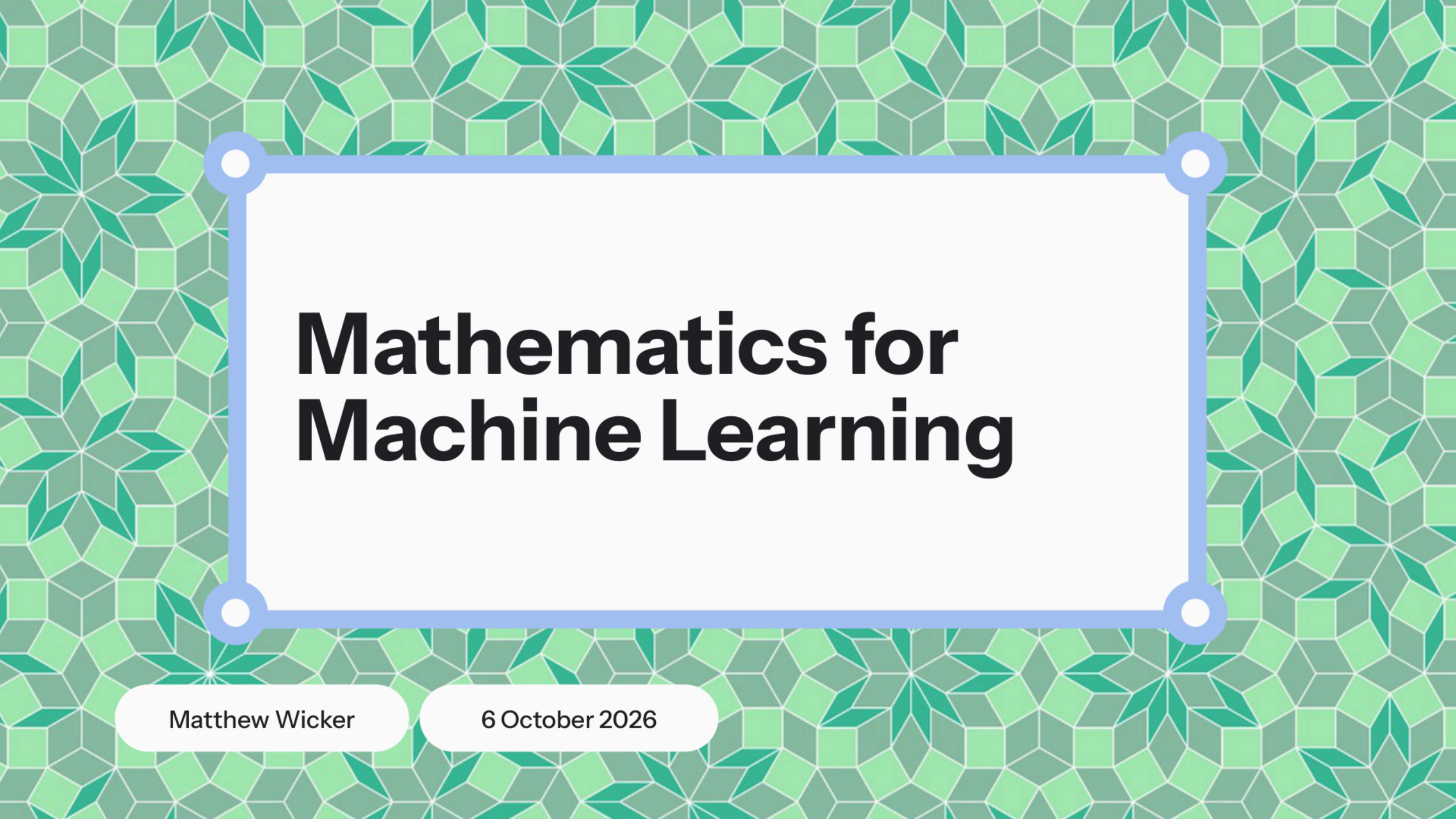


The background of the slide is a repeating geometric pattern in shades of green, featuring a mix of hexagons and star-like shapes. A light blue rectangular frame with rounded corners and white circular accents at the corners is centered on the slide.

Mathematics for Machine Learning

Matthew Wicker

6 October 2026

Today's Lecture

01

Course Overview &
Course Philosophy

02

Outlining Machine
Learning Problem
Settings

03

Developing and
learning in linear
models

Course Information

Lecture Times:

- Tuesdays from 9am - 11am
- Thursdays from 9am - 11am

Office Hours: Thursdays from 2pm - 4pm

Today:

- 2 hours of lecture material

Course Information

Lecture Times:

- Tuesdays from 9am – 11am
- Thursdays from 9am – 11am

Office Hours: Thursdays from 2pm – 4pm

Normally:

- 1 – 1.5 hours of lecture material
- 0.5 – 1 hour of exam problems

Course Materials

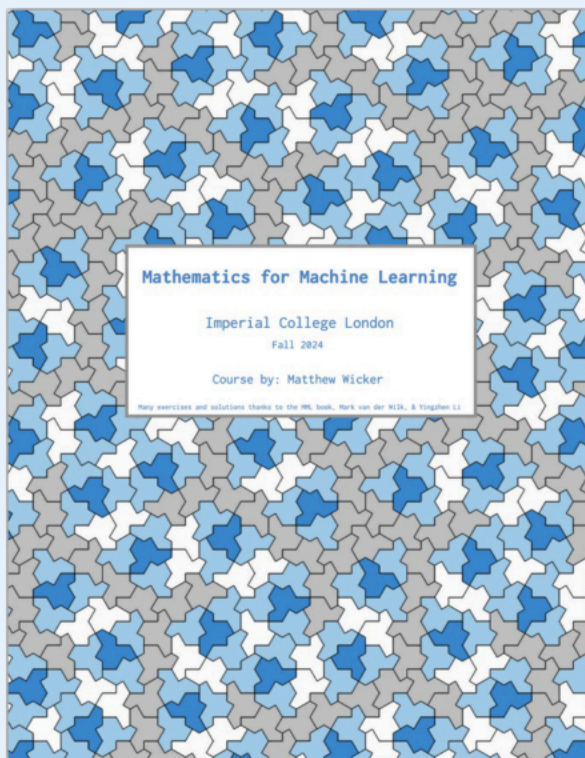
From Me, Every Lecture:

- Lecture Notes
- A Practice Exam Question (12 total)
- Practice Exercises
- Practice Exercise Solutions
- Recorded Lecture

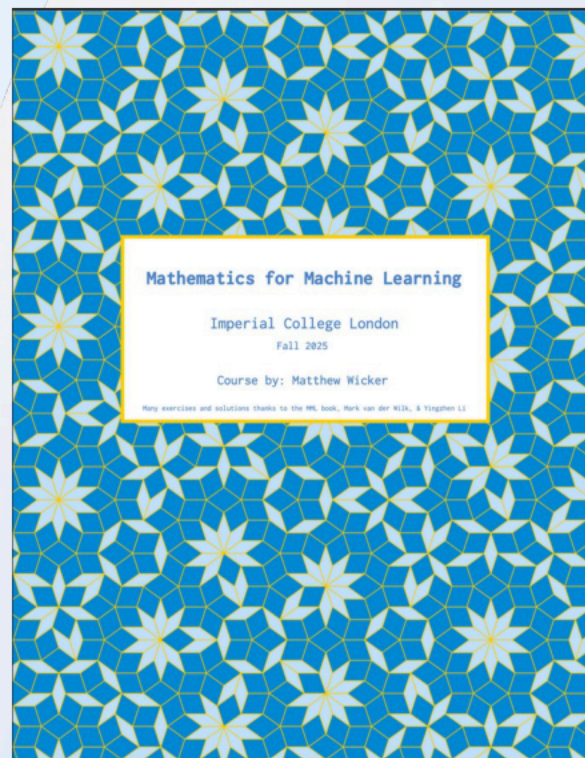
From Me, At Course End:

- Completed course book
- 4x Practice Exams + Model Solutions

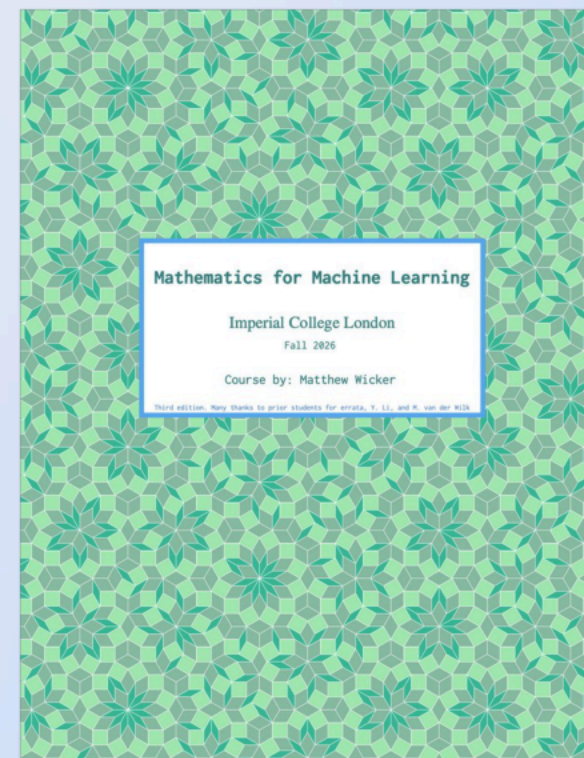
Course Materials



2024 Edition



2025 Edition



2026 Edition

Course Philosophy

Previous ML Courses cover:

What is ML? How do we do it?

This course:

Why do we do ML that way?

Course Philosophy

Previous ML Courses cover:

What is ML? How do we do it?

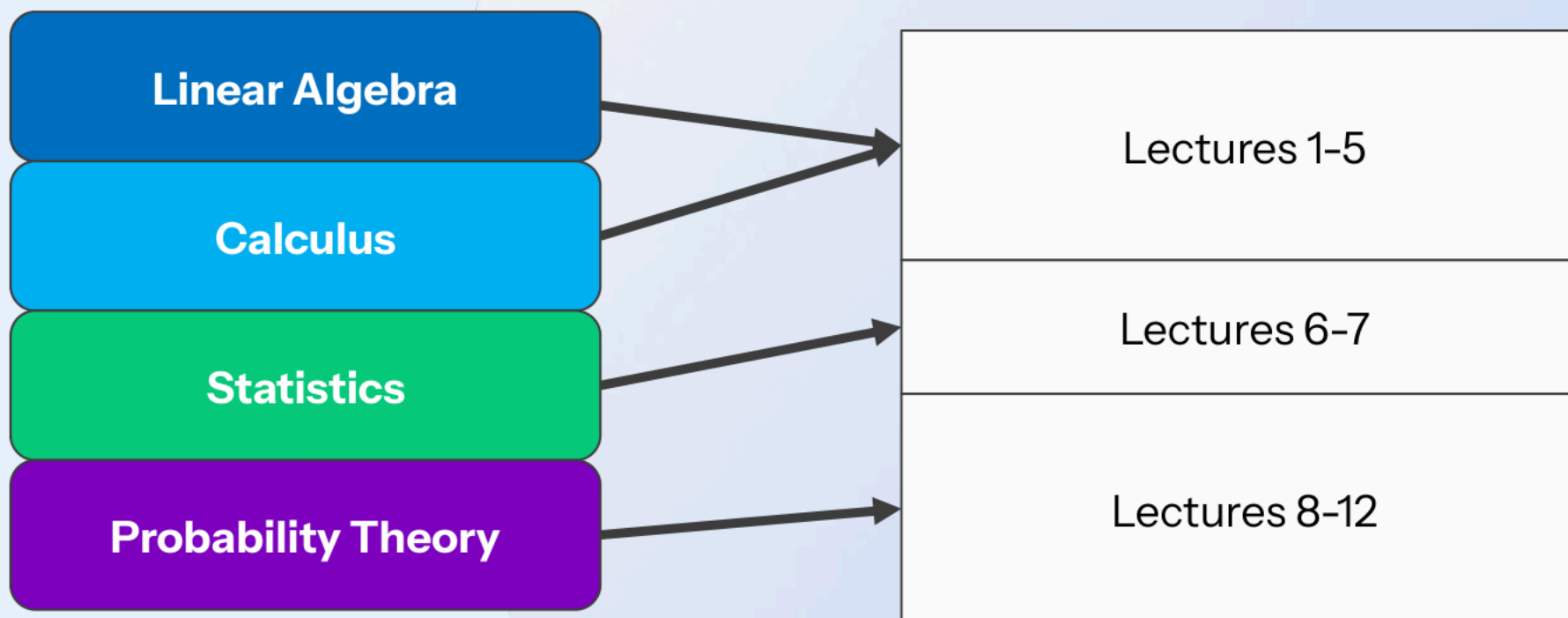
This course:

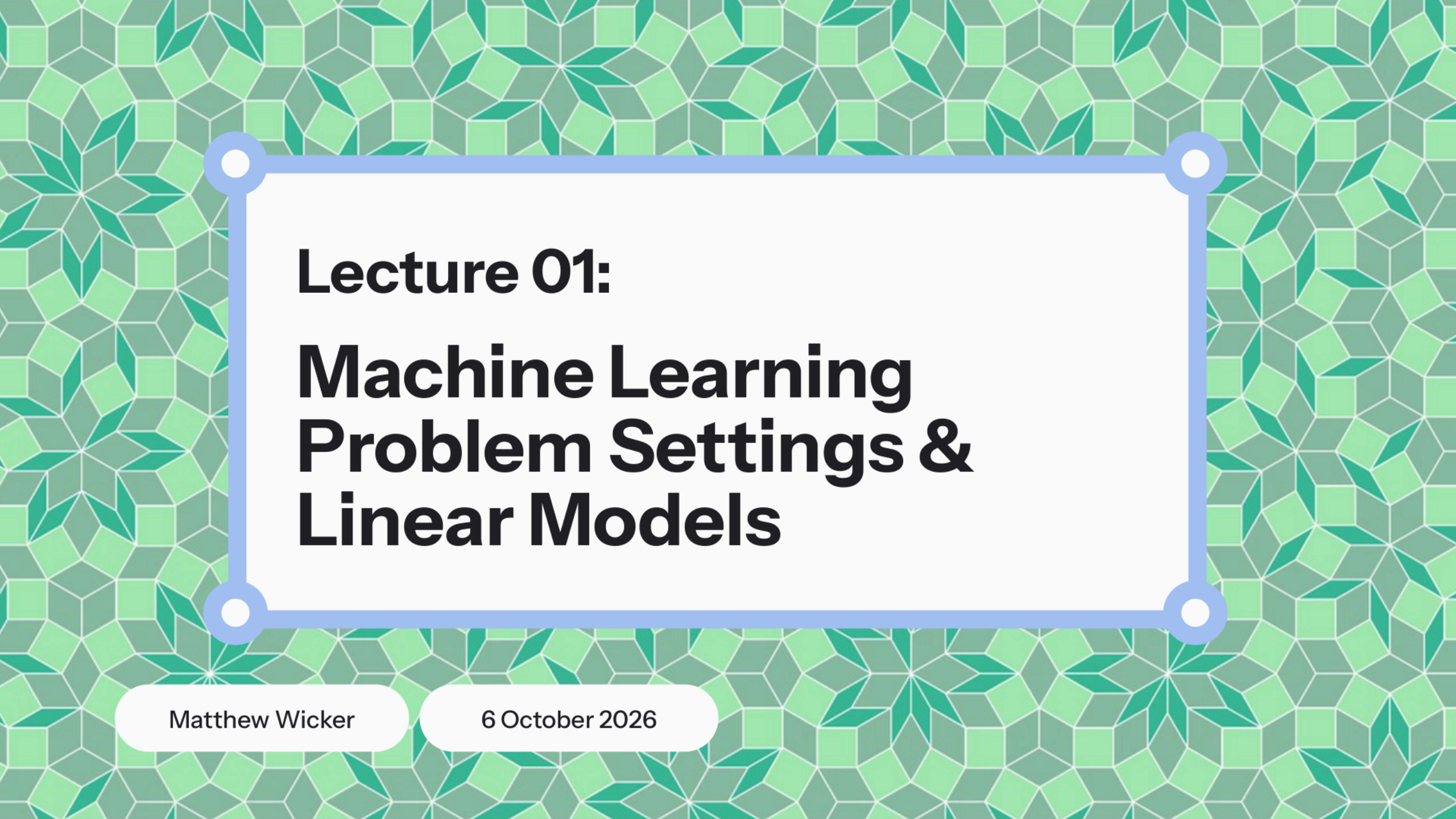
Why do we do ML that way?

Viewing machine learning through a mathematical lens allows us to appreciate the foundational philosophy of the discipline, and these mathematical techniques are the primary focus of this course

Course Structure

The course (and the exam) will be relatively challenging. In conversation with previous students we made lots of changes to help with this:



The background of the slide is a repeating geometric pattern in various shades of green. The pattern consists of interconnected polygons, some of which are stylized to resemble leaves or star shapes. A light blue rectangular frame with rounded corners and white circular accents at the corners is positioned in the center of the slide.

Lecture 01:

Machine Learning Problem Settings & Linear Models

Matthew Wicker

6 October 2026

A) Machine learning is the study of algorithms which automatically perform a task by processing an example dataset.

B) Machine learning is a computational approach that allows systems to automatically learn and improve from experience without being explicitly programmed.

A) Machine learning is the study of **algorithms** which automatically perform a **task** by processing an **example dataset**.

B) Machine learning is a computational approach that allows systems to automatically learn and improve from experience without being explicitly programmed.

A) Machine learning is the study of **algorithms** which automatically perform a **task** by processing an **example dataset**.

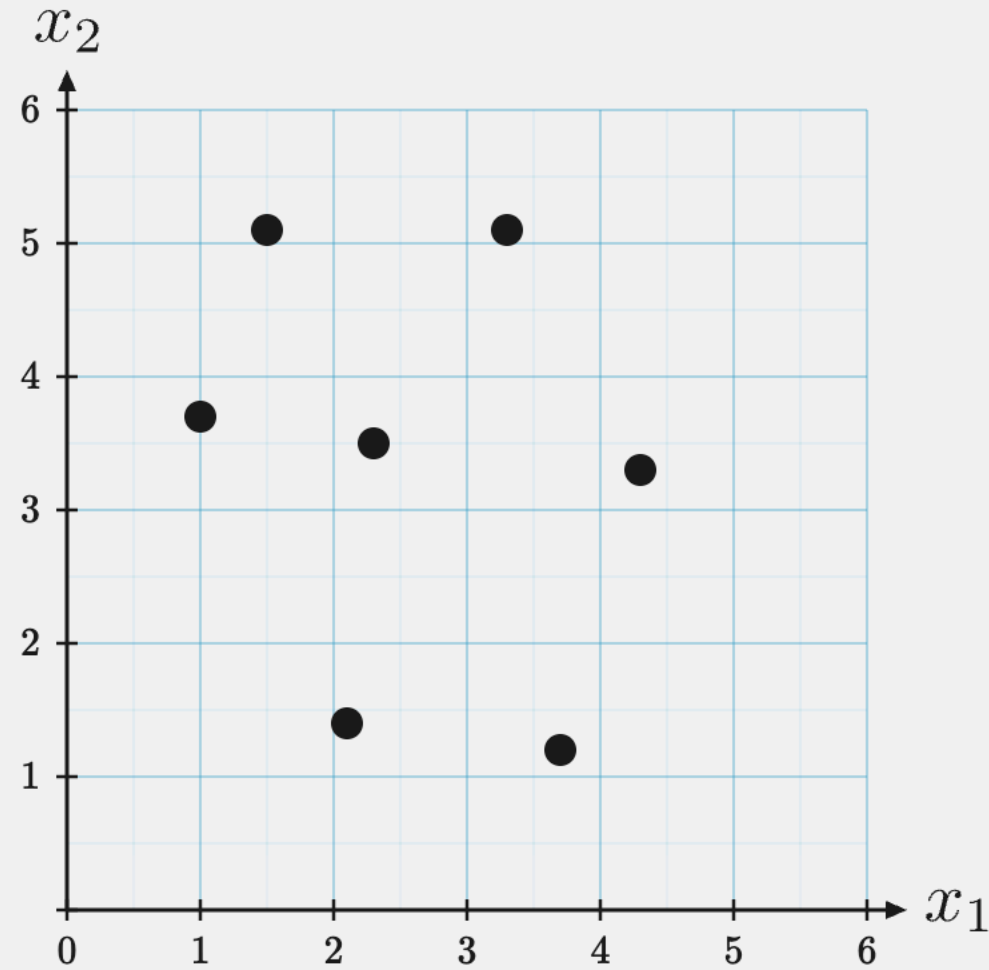
B) Machine learning is a computational approach that allows systems to automatically **learn** and improve from **experience** without being **explicitly programmed**.

A) Machine learning is the study of **algorithms** which automatically perform a **task** by processing an **example dataset**.

B) Machine learning is a computational approach that allows systems to automatically **learn** and improve from **experience** without being **explicitly programmed**.

Note: Sometimes I use terms that perhaps you have not heard in prior math or machine learning courses, at the end of lecture in my feedback form you can tell me which terms were unfamiliar and I will explicitly cover them next lecture

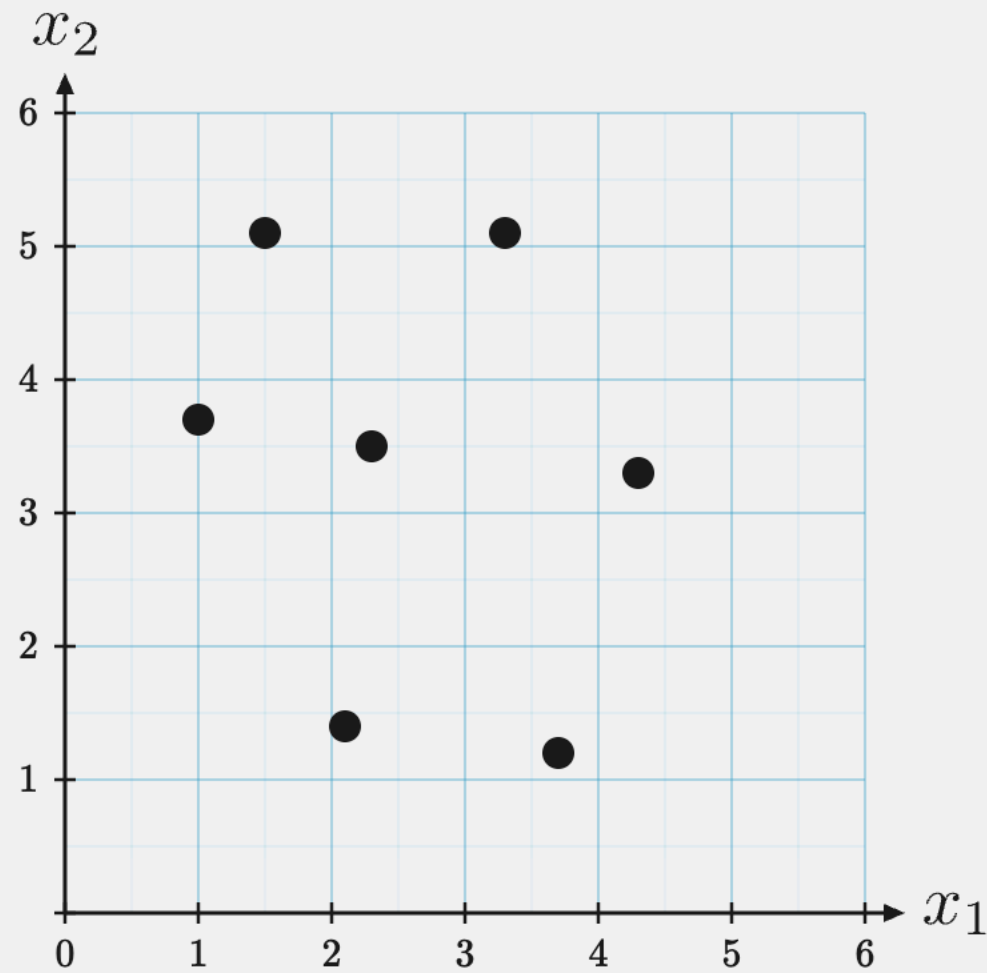
Mathematically, what describes this data:



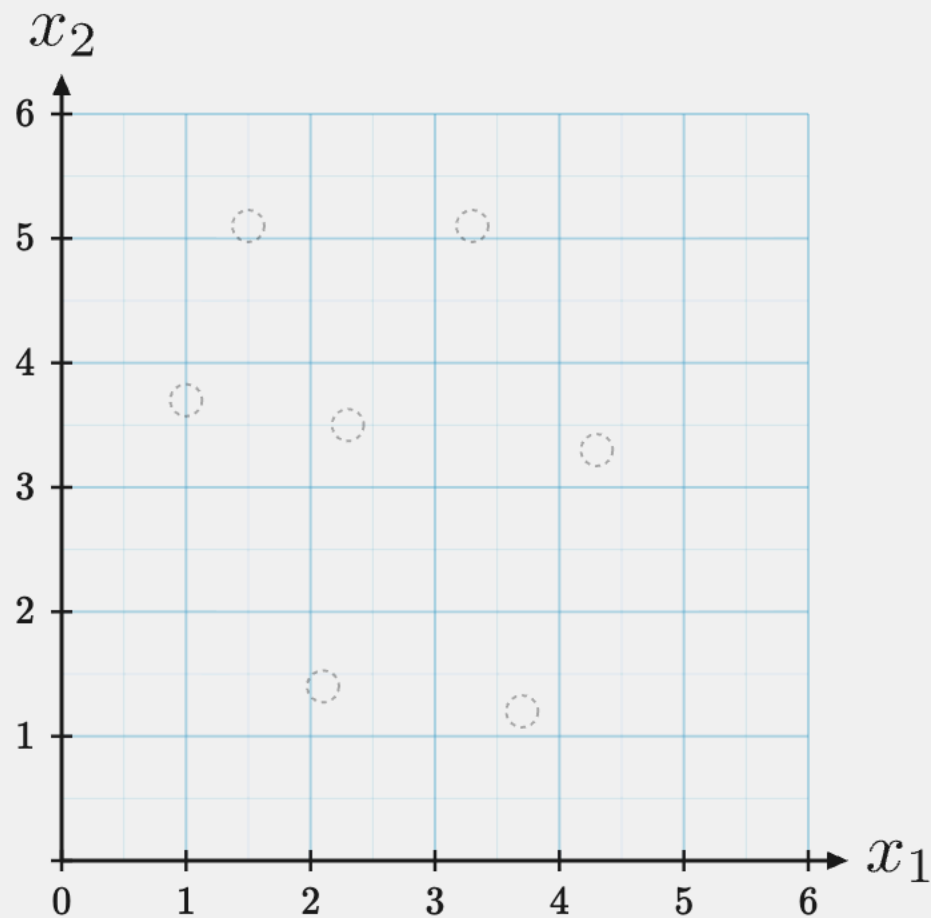
A quick primer on notational conventions:

- x : real-valued scalars.
- $\mathbf{x}, \boldsymbol{\mu}$: deterministic vectors.
- $\mathbf{x}$: random variables (vector-valued).
- $\mathbf{X}$: deterministic matrices.
- $\mathbf{X}$: random matrices.
- θ : represents the collection of all model parameters.

Mathematically, what describes this data:



A Set-Theoretic Description:



$$\left\{ \begin{array}{ccccccc} \bullet & , & \bullet & , & \bullet & , & \bullet & , & \bullet & , & \bullet & , & \bullet \\ \mathbf{x}^{(1)} & & \mathbf{x}^{(2)} & & \mathbf{x}^{(3)} & & \mathbf{x}^{(4)} & & \mathbf{x}^{(5)} & & \mathbf{x}^{(6)} & & \mathbf{x}^{(7)} \end{array} \right\}$$

$$\{\mathbf{x}^{(i)}\}_{i=1}^N$$

A Set-Theoretic Description:

$$\{\mathbf{x}^{(i)}\}_{i=1}^N$$

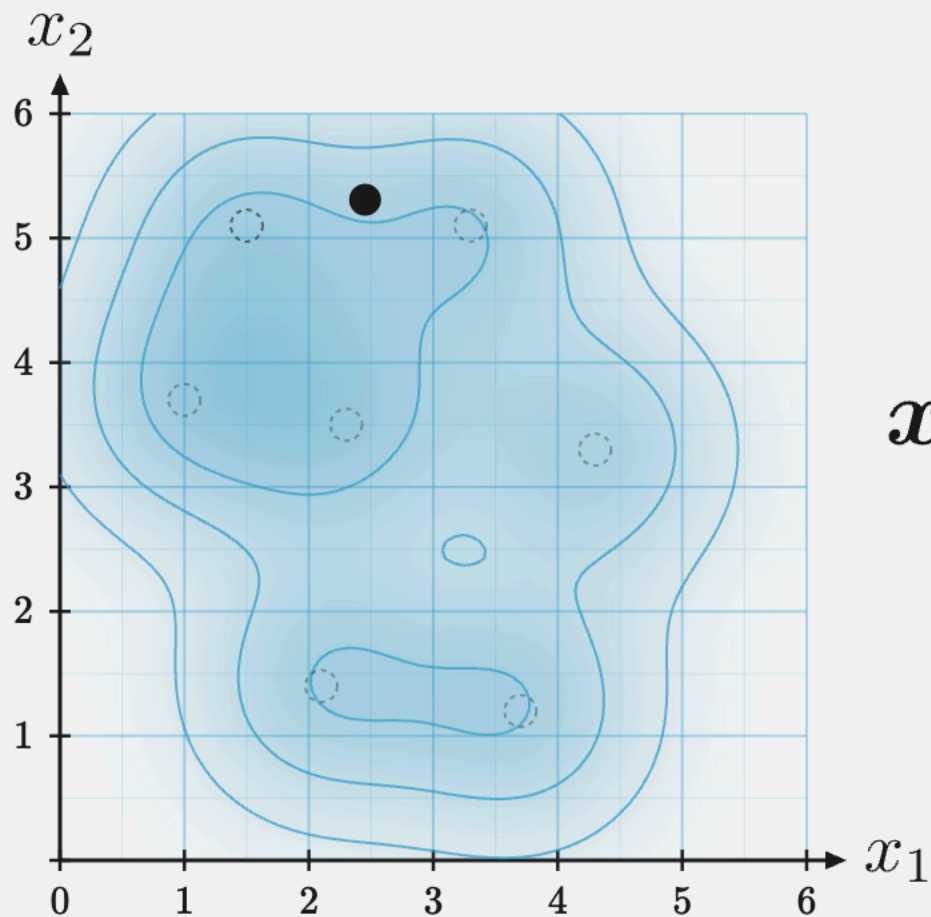
$$\{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^N$$

$$\{(\mathbf{s}^{(i)}, \mathbf{a}^{(i)}, \mathbf{s}^{(i+1)})\}_{i=1}^N$$

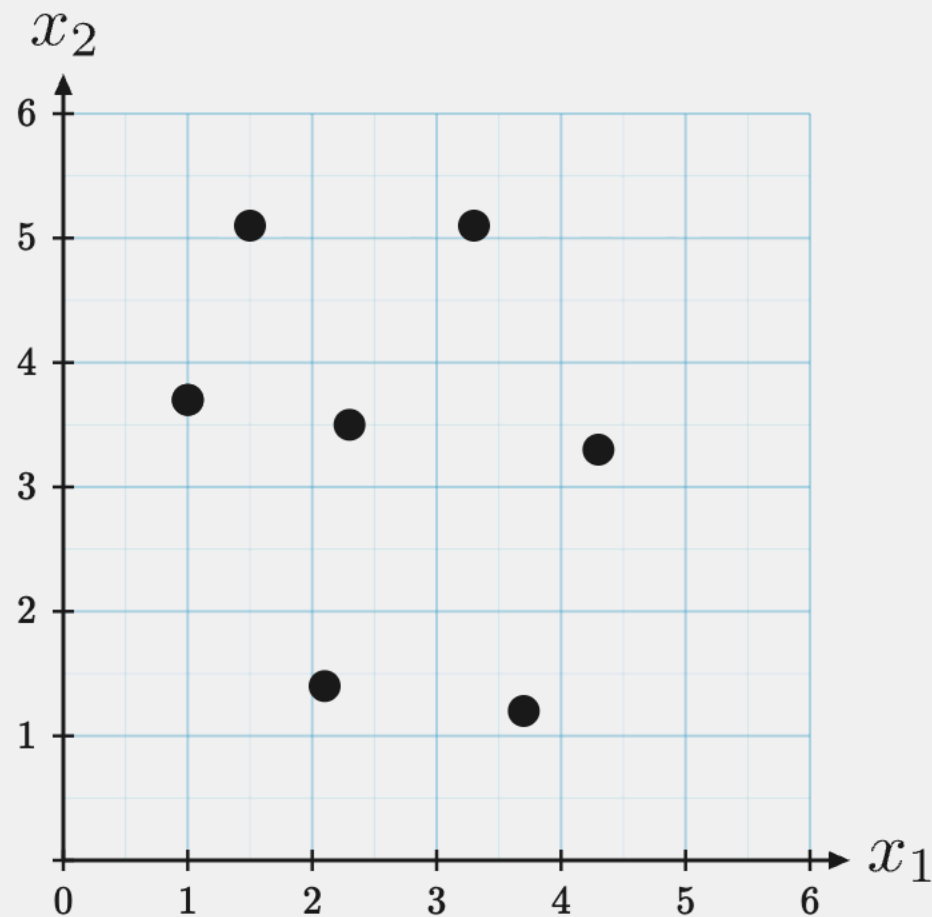
As an abstraction, sets capture many intuitive properties of datasets:

- Order/permutation invariant
- Clarifies what info. we have access to
- Size of set $\rightarrow$ sample complexity
- Allows for abstract reasoning about feature/label space

A Probabilistic Description:



$$\mathbf{x} \sim P(\mathbf{X})$$



A Probabilistic Description:

Viewing data as samples from a probability distribution also clarifies what observations we have access to and what we suggests many practical tools for reasoning about the problem at hand.

$$\mathbf{x} \sim P(\mathbf{X})$$

$$(\mathbf{x}, \mathbf{y}) \sim P(\mathbf{X}, \mathbf{Y})$$

$$\mathbf{s}^{(i+1)} \sim P(\mathbf{s}^{(i+1)} \mid \mathbf{s}^{(i)}, \mathbf{a}^{(i)})$$

A Probabilistic Description:

As we will see one critical aspect/assumption we make of our data is the i.i.d. assumption:

Independence:

Identically Distributed:

A Probabilistic Description:

As we will see one critical aspect/assumption we make of our data is the i.i.d. assumption:

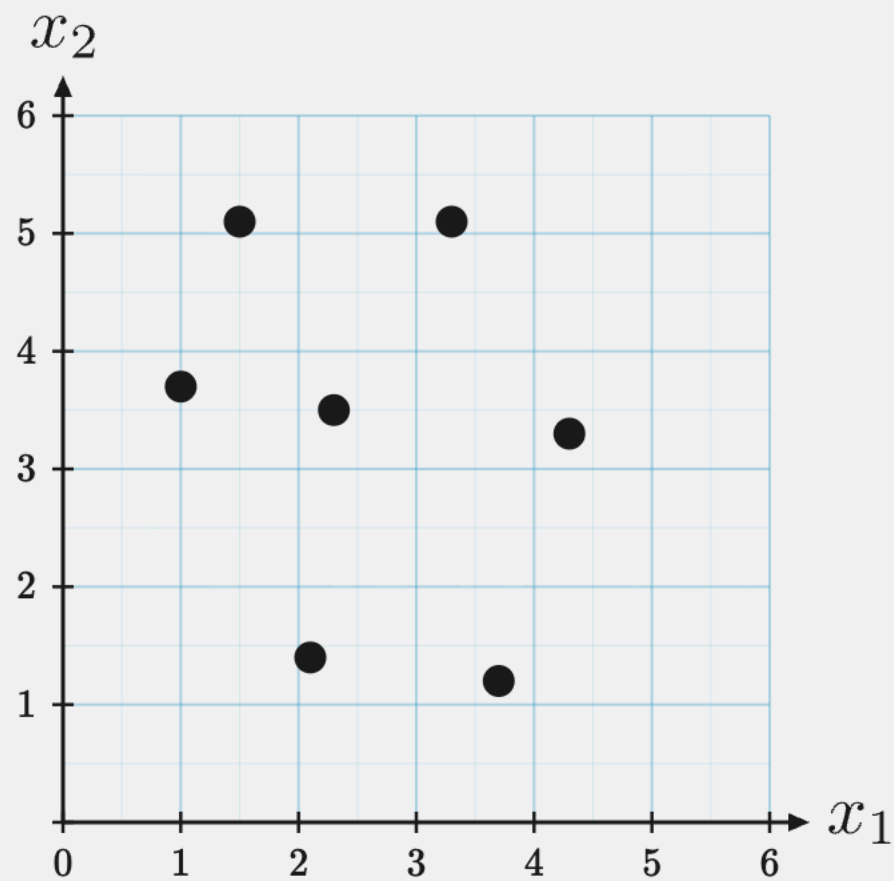
Independence:

$$\begin{aligned} P(\mathbf{x}^{(i_1)} = \mathbf{x}^{(i_1)}, \mathbf{x}^{(i_2)} = \mathbf{x}^{(i_2)}, \dots, \mathbf{x}^{(i_k)} = \mathbf{x}^{(i_k)}) \\ = P(\mathbf{x}^{(i_1)} = \mathbf{x}^{(i_1)}) \cdot P(\mathbf{x}^{(i_2)} = \mathbf{x}^{(i_2)}) \cdot \dots \cdot P(\mathbf{x}^{(i_k)} = \mathbf{x}^{(i_k)}) \end{aligned}$$

Identically Distributed:

$$f_{\mathbf{x}^{(i_1)}}(\mathbf{x}) = f_{\mathbf{x}^{(i_2)}}(\mathbf{x}) = \dots = f_{\mathbf{x}^{(i_k)}}(\mathbf{x}) = f_{\mathbf{x}}(\mathbf{x})$$

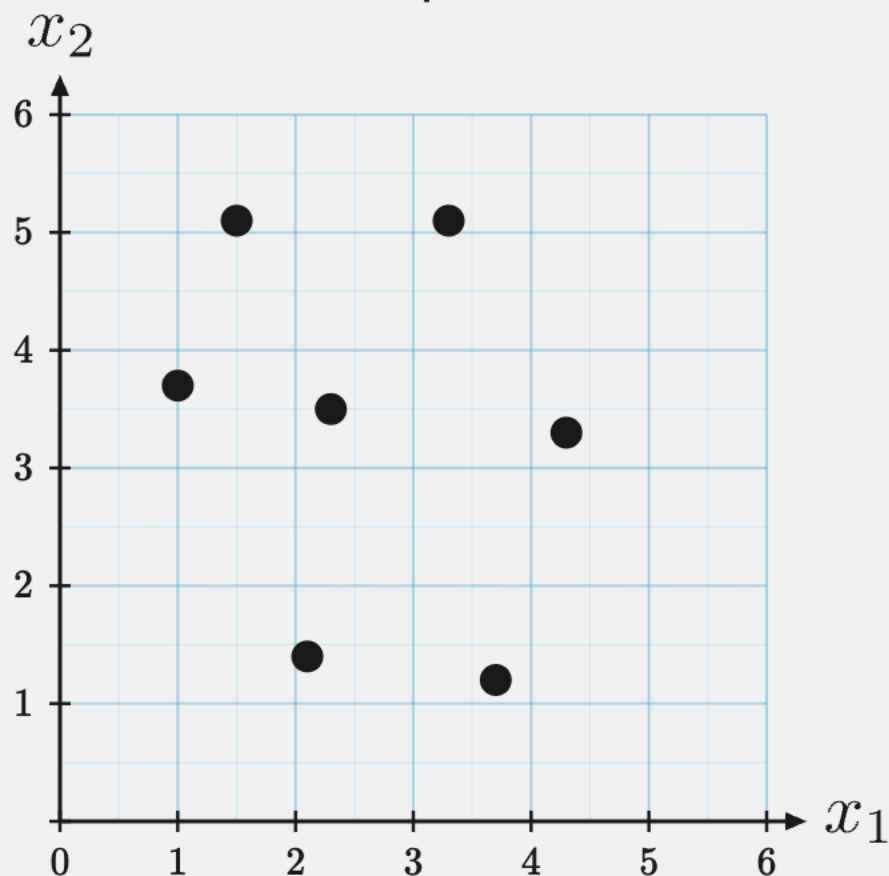
A Linear Algebra Description:



$$\begin{aligned} \mathbf{x}^{(1)\top} &= \begin{bmatrix} 1.0 & 3.7 \end{bmatrix} \\ \mathbf{x}^{(2)\top} &= \begin{bmatrix} 1.5 & 5.1 \end{bmatrix} \\ \mathbf{x}^{(3)\top} &= \begin{bmatrix} 2.1 & 1.4 \end{bmatrix} \\ \mathbf{x}^{(4)\top} &= \begin{bmatrix} 2.3 & 3.5 \end{bmatrix} \\ \mathbf{x}^{(5)\top} &= \begin{bmatrix} 3.3 & 5.1 \end{bmatrix} \\ \mathbf{x}^{(6)\top} &= \begin{bmatrix} 3.7 & 1.2 \end{bmatrix} \\ \mathbf{x}^{(7)\top} &= \begin{bmatrix} 4.3 & 3.3 \end{bmatrix} \end{aligned}$$

A Linear Algebra Description:

Like the others, this perspective empowers us to use the tools of linear algebra on our features. For example, what would it mean if the column space of our matrix was not full rank?

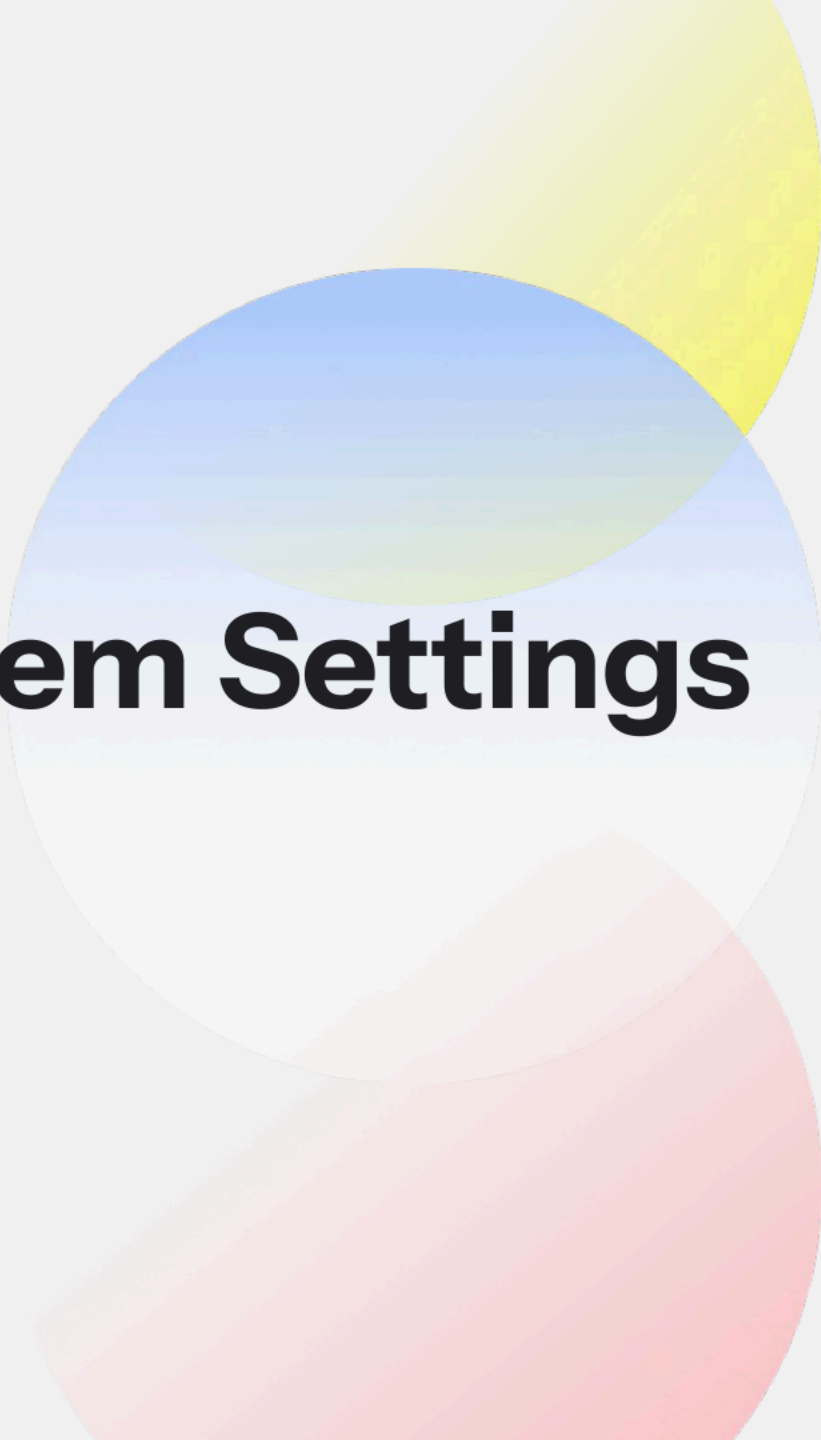


$$X = \begin{bmatrix} 1.0 & 3.7 \\ 1.5 & 5.1 \\ 2.1 & 1.4 \\ 2.3 & 3.5 \\ 3.3 & 5.1 \\ 3.7 & 1.2 \\ 4.3 & 3.3 \end{bmatrix} \begin{matrix} x^{(1)\top} \\ x^{(2)\top} \\ x^{(3)\top} \\ x^{(4)\top} \\ x^{(5)\top} \\ x^{(6)\top} \\ x^{(7)\top} \end{matrix}$$

A Linear Algebra Description:

Standard operations on matrices are then given new potential interpretations, e.g., X having non-full column space implies feature redundancy:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^{(1)\top} \\ \vdots \\ \mathbf{x}^{(m)\top} \end{bmatrix} = \begin{bmatrix} \mathbf{x}_1^{(1)\top}, & \mathbf{x}_2^{(1)\top}, & \dots, & \mathbf{x}_n^{(1)\top} \\ \vdots, & \vdots, & \ddots, & \vdots \\ \mathbf{x}_1^{(m)\top}, & \mathbf{x}_2^{(m)\top}, & \dots, & \mathbf{x}_n^{(m)\top} \end{bmatrix}$$

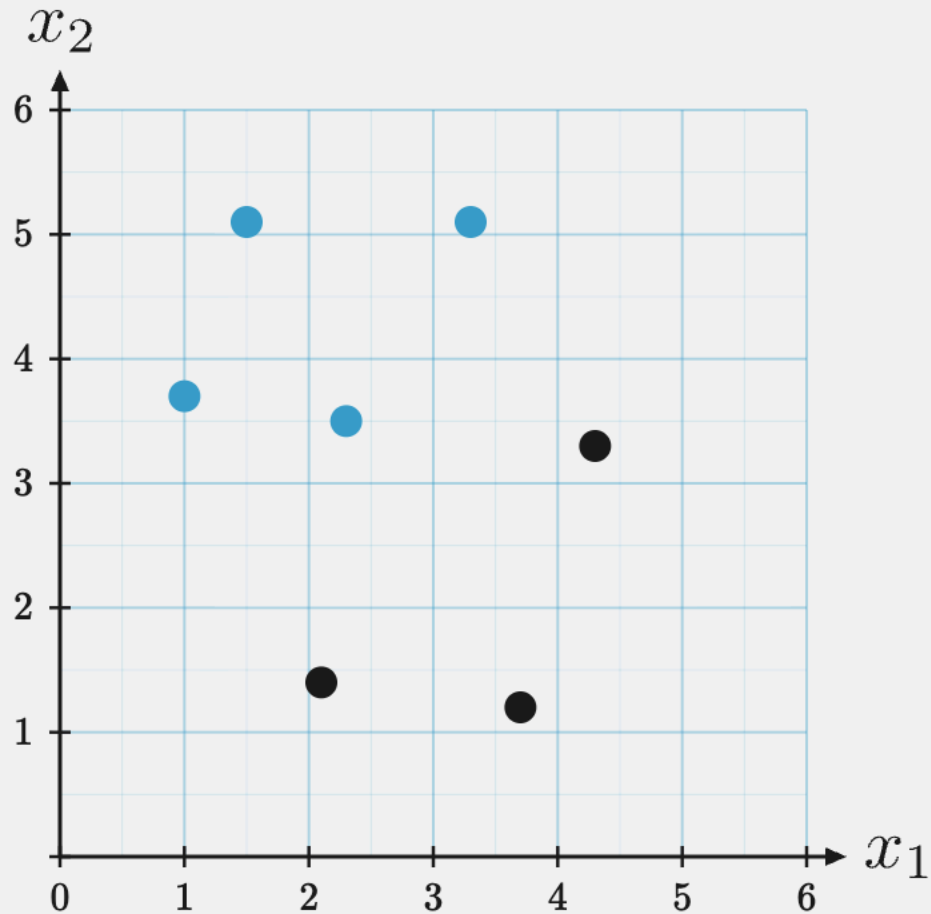


Machine Learning Problem Settings

Supervised Learning

When we have some supervisory signal (example of what we want) we encounter the most "traditional" examples of machine learning.

Supervised Learning



Set

$$\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$$

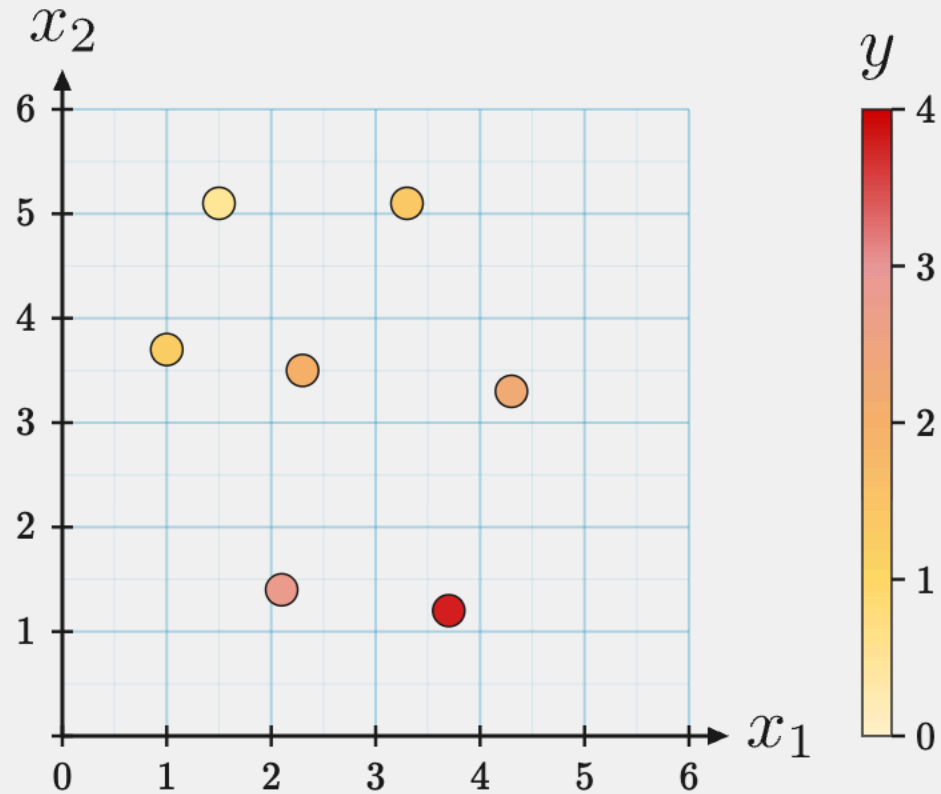
Probability

$$(\mathbf{x}, y) \sim P(\mathbf{X}, \mathbf{Y})$$

Linear algebra

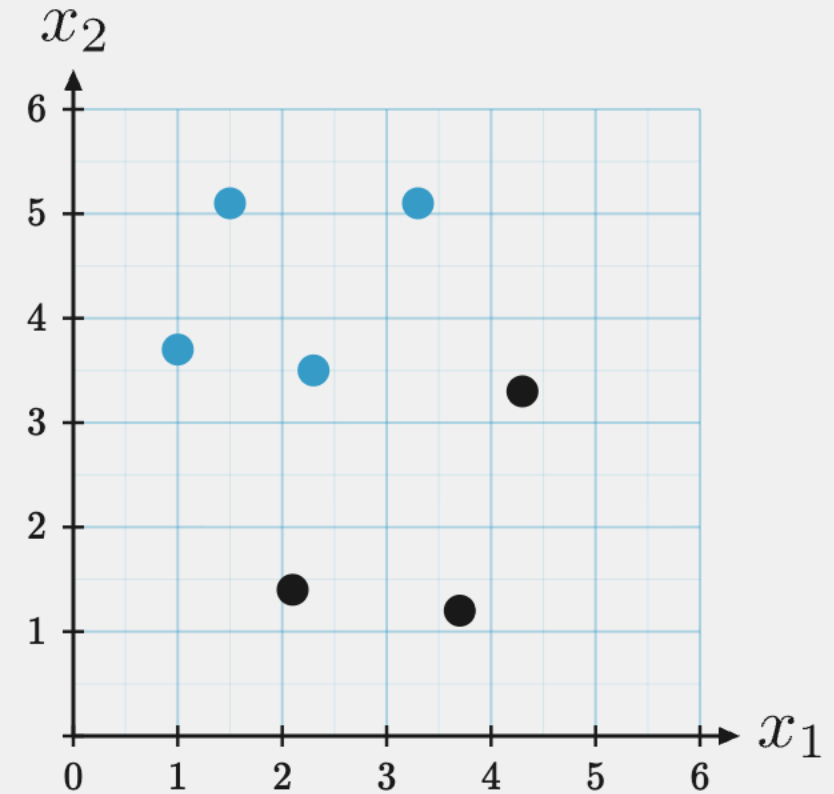
$$\mathbf{X} = \begin{bmatrix} 1.0 & 3.7 \\ 1.5 & 5.1 \\ 2.1 & 1.4 \\ 2.3 & 3.5 \\ 3.3 & 5.1 \\ 3.7 & 1.2 \\ 4.3 & 3.3 \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

Supervised Learning



Supervised (Regression)

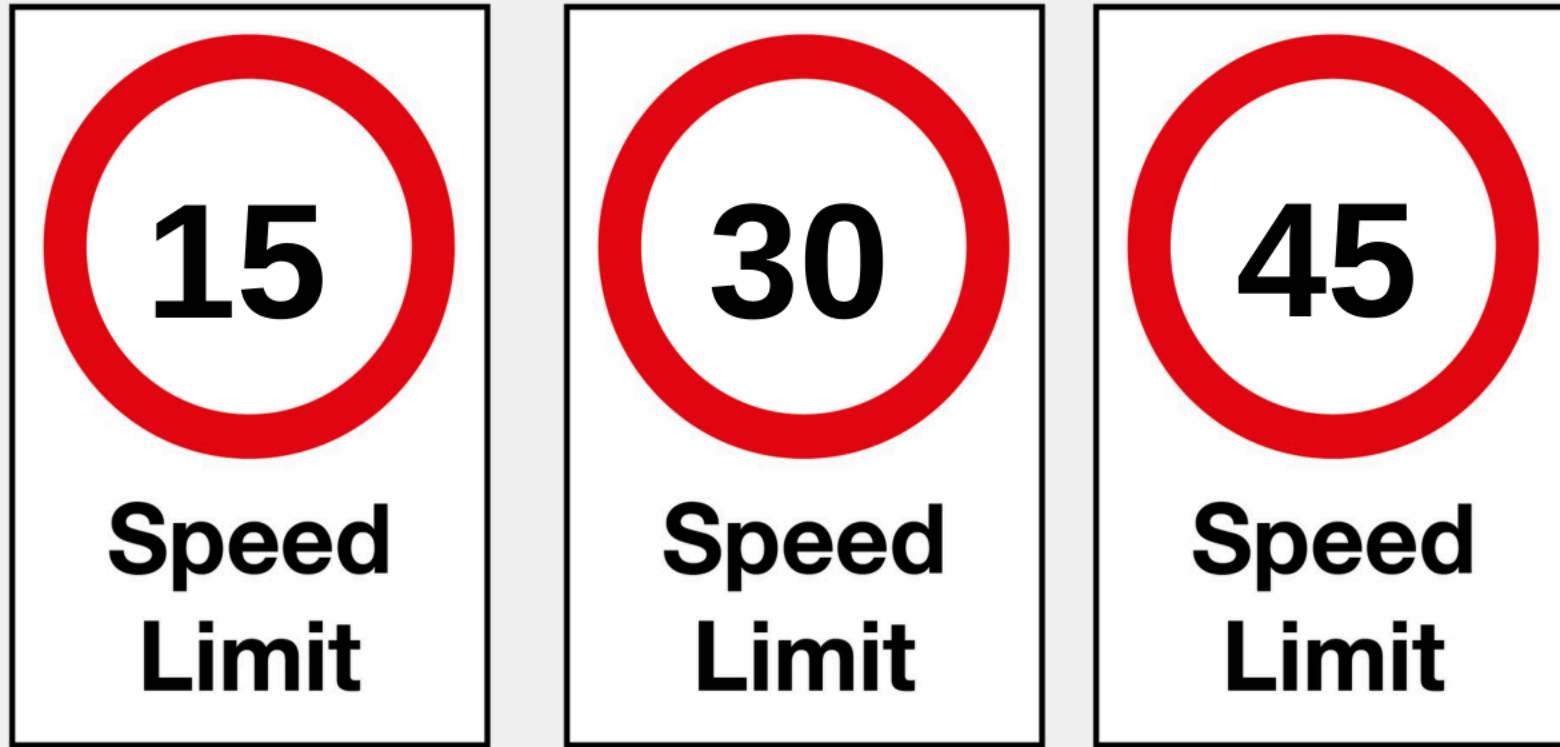
$$\mathbf{y}^\top = \begin{bmatrix} 1.2 & 0.5 & 2.8 & 2.0 & 1.4 & 3.8 & 2.2 \end{bmatrix}$$



Supervised (Classification)

$$\mathbf{y}^\top = \begin{bmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

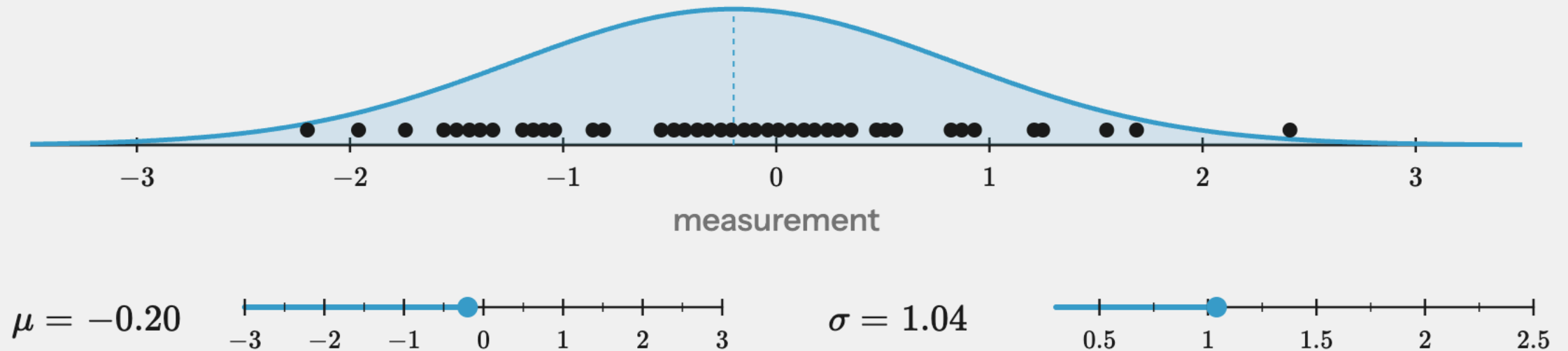
Supervised Learning



Unsupervised Learning

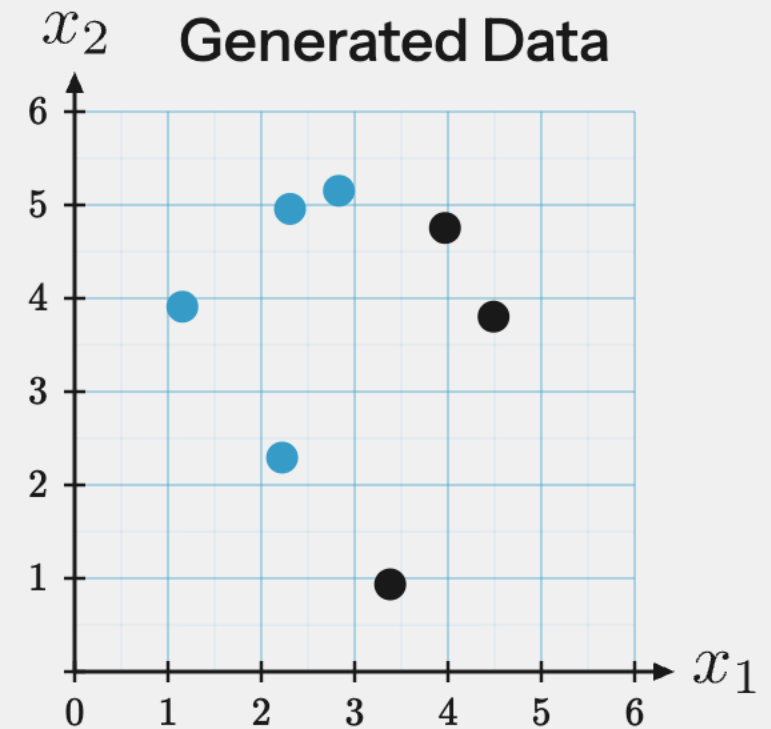
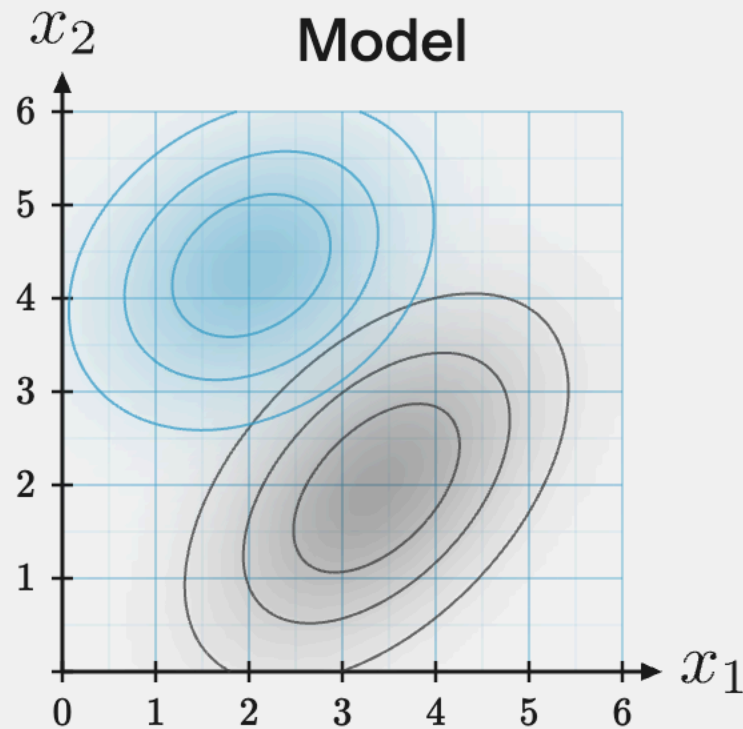
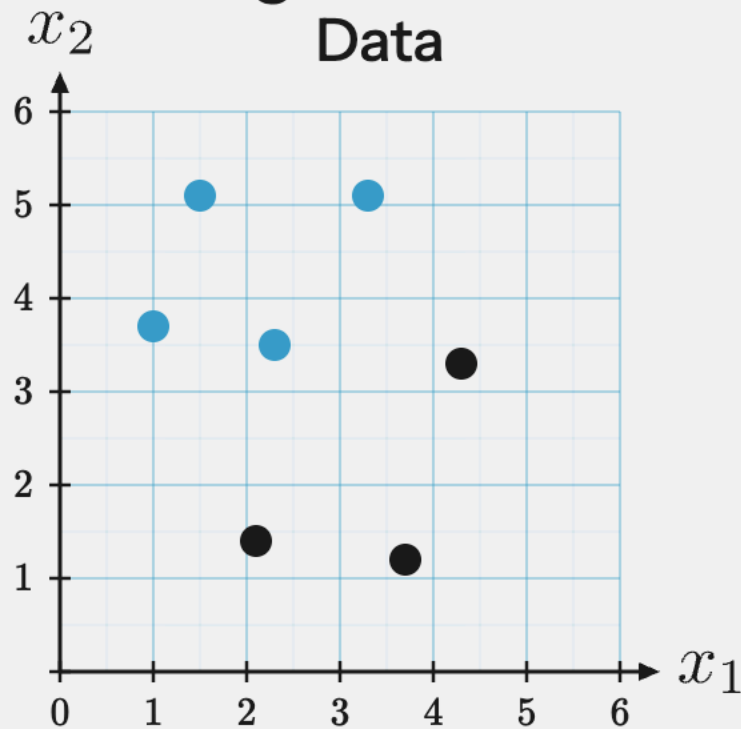
Also known as knowledge discovery, unsupervised learning methods help us discover patterns between data points.

$$\log L(\mu, \sigma) = -61.2$$



Unsupervised Learning - Generative

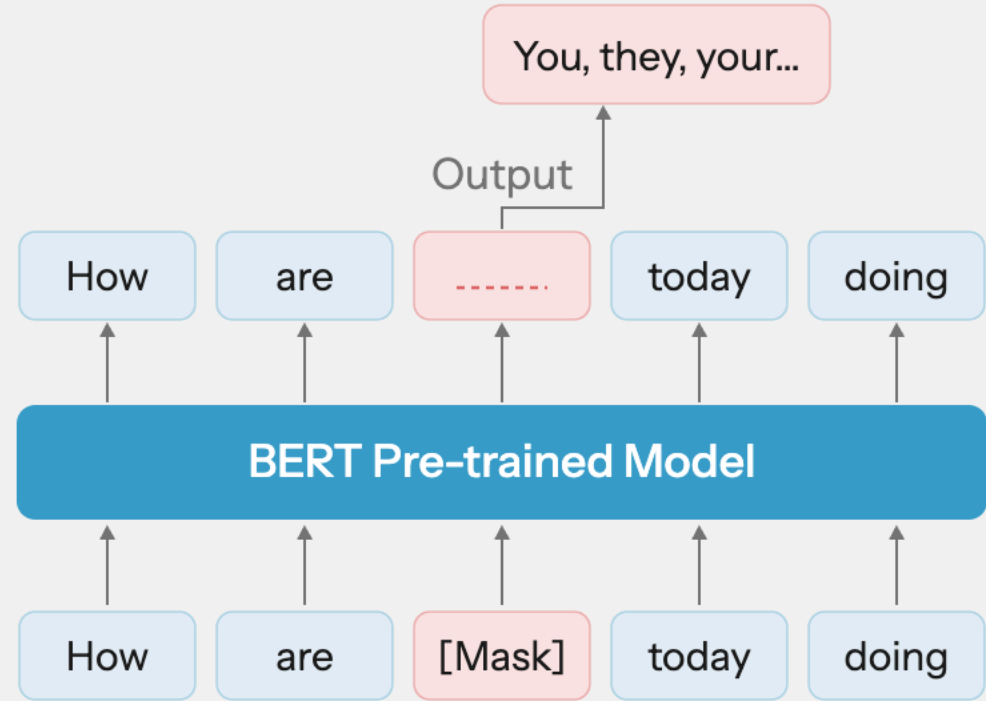
Generative learning may fall (not cleanly) under unsupervised learning.



Hybrid Settings

Some settings cannot be neatly described in a single setting or combine multiple different settings to accomplish a task.

A great example is pre- and post-training regimes for natural language generation



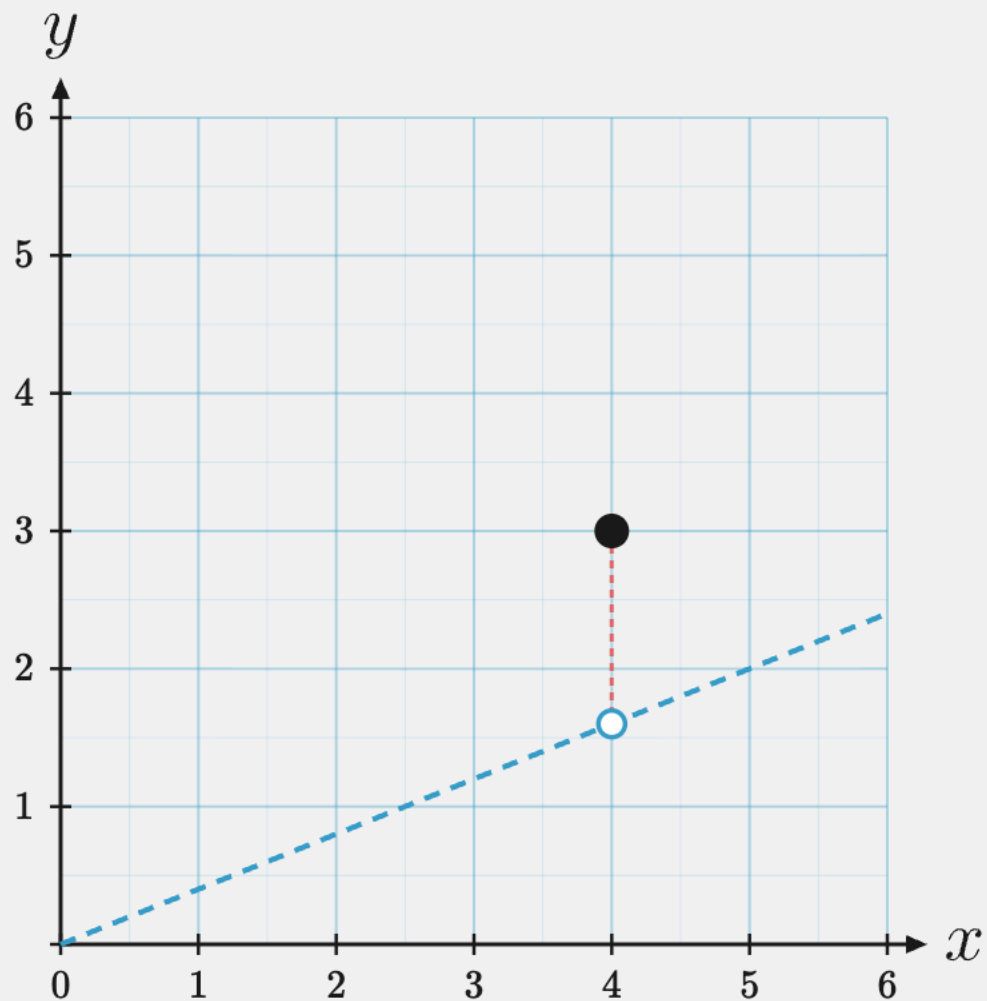
The Linear Model

A) Machine learning is the study of *algorithms* which automatically perform a task by processing an example dataset.

algorithms are not the same as *models*

Instead, models are *assumed* and algorithms return learned models. Before we are ready to look at and consider learning algorithms, let's observe a *model*.

The Linear Model

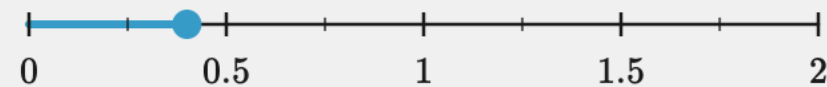


$$\hat{y} = w x$$

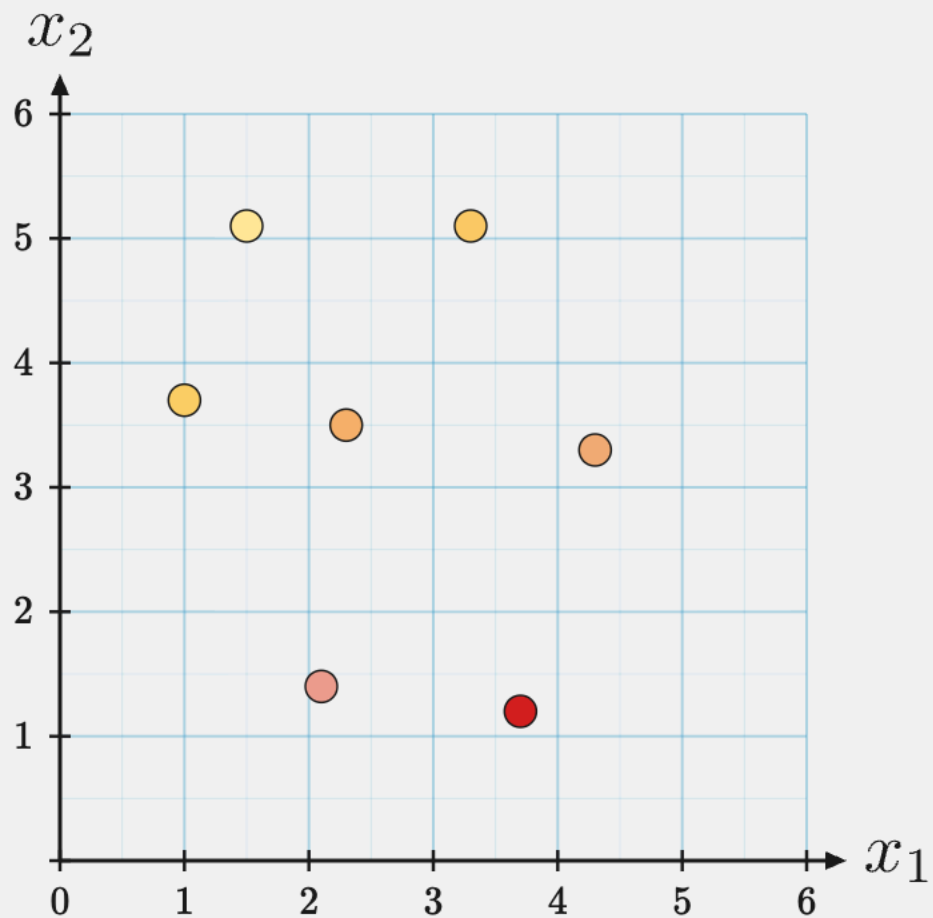
$$\hat{y} = w x = 0.40 \times 4.0 = 1.60$$

$$y = 3.0$$

$$w = 0.40$$



The Linear Model



$$\mathbf{X} = \begin{bmatrix} 1.0 & 3.7 \\ 1.5 & 5.1 \\ 2.1 & 1.4 \\ 2.3 & 3.5 \\ 3.3 & 5.1 \\ 3.7 & 1.2 \\ 4.3 & 3.3 \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} 1.2 \\ 0.5 \\ 2.8 \\ 2.0 \\ 1.4 \\ 3.8 \\ 2.2 \end{bmatrix}$$

Building up some math

We can start with the most basic unit, a real valued variable.

$$x \in \mathbb{R}$$

We generalize this to n dimensions using a vector (for which we will use subscripts)

$$\boldsymbol{x} \in \mathbb{R}^n$$

Vectors are by default column vectors and thus you can express their shape as having only one column

$$\boldsymbol{x} \in \mathbb{R}^{n \times 1}$$

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$$

Building up some math

We generalize vectors into matrices by allowing there to be m -many rows (for example) and then the matrix can be indexed with two values, by convention i and j

$$\mathbf{X} \in \mathbb{R}^{n \times m}$$

$$\mathbf{X} = \begin{bmatrix} \mathbf{X}_{1,1}, \mathbf{X}_{1,2}, \dots, \mathbf{X}_{1,m}, \\ \mathbf{X}_{2,1}, \mathbf{X}_{2,2}, \dots, \mathbf{X}_{2,m}, \\ \vdots, \ddots, \vdots \\ \mathbf{X}_{n,1}, \mathbf{X}_{n,2}, \dots, \mathbf{X}_{n,m} \end{bmatrix}$$

Building up some math

We will also further generalize to tensors which have multiple dimensions and have a single index per dimension.

With two matrices, multiplication is well defined

With tensors we need to define which dimension is "contracted"

$$\mathbf{A} \in \mathbb{R}^{n \times m \times l}$$

$$\mathbf{A}_{i,j,k} \in \mathbb{R}$$

Building up some math (Summation Notation)

We will also further generalize to tensors which have multiple dimensions and have a single index per dimension.

With two matrices, multiplication is well defined

With tensors we need to define which dimension is "contracted"

$$\mathbf{A} \in \mathbb{R}^{n \times m}, \mathbf{B} \in \mathbb{R}^{m \times l}$$

$$C_{i,j} = \sum_{k=1}^m A_{i,k} B_{k,j}$$

$$\mathbf{AB} \in \mathbb{R}^{n \times l}$$

Building up some math (Summation Notation)

We will also further generalize to tensors which have multiple dimensions and have a single index per dimension.

With two matrices, multiplication is well defined

$$C_{i,j} = \sum_{k=1}^m A_{i,k} B_{k,j}$$

With tensors we need to define which dimension is "contracted"

$$A \in \mathbb{R}^{n \times m \times l} \quad B \in \mathbb{R}^{l \times o}$$
$$C_{i,j,u} = \sum_{k=1}^l A_{i,j,k} B_{k,o}$$

Building up some math (Einstein Notation)

Einstein notation (which I will try to avoid where possible in favor of summation notation) drops the summation.

A few basic rules:

- * if an index appears once, it is not summed over
- * if an index appears twice it is summed over
- * no index should appear more than twice

More rules to familiarize yourself with are referenced in the lecture notes

$$C_{i,j} = A_{i,k} B_{k,j}$$

$$\mathbf{A} \in \mathbb{R}^{n \times m \times l} \quad \mathbf{B} \in \mathbb{R}^{l \times o}$$

$$C_{i,j,u} = A_{i,j,k} B_{k,o}$$

Building up some math

Connecting to PyTorch, you can also express anything in Einstein notation as follows:

Operation	Einstein form	PyTorch
Matrix multiplication	$C_{ij} = A_{ik} B_{kj}$	<code>torch.einsum('ik,kj->ij', A, B)</code>
Dot product	$c = a_i b_i$	<code>torch.einsum('i,i->', a, b)</code>
Outer product	$C_{ij} = a_i b_j$	<code>torch.einsum('i,j->ij', a, b)</code>
Matrix–vector product	$y_i = A_{ij} x_j$	<code>torch.einsum('ij,j->i', A, x)</code>
Tensor contraction	$C_{i,j,o} = A_{i,j,k} B_{k,o}$	<code>torch.einsum('ijk,ko->ijo', A, B)</code>

Table 1: Common tensor operations in Einstein notation and PyTorch’s `einsum` syntax.

Modelling Decisions and Implications

A linear model:

$$\hat{y} = \boldsymbol{w}^\top \boldsymbol{x} := \sum_i w_i x_i$$

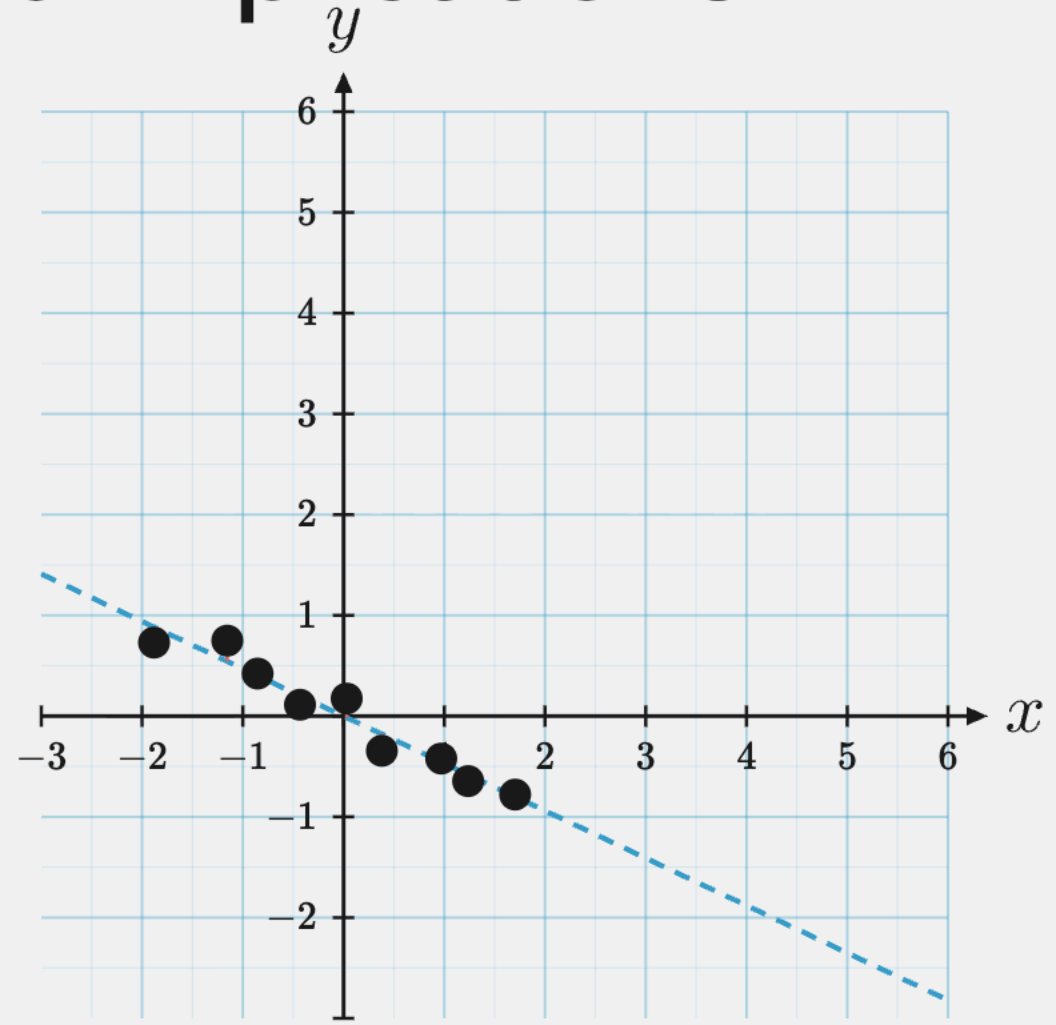
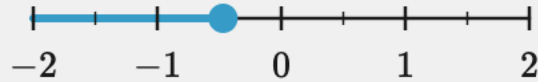
Definition 1.3.1. Linear transformation A linear transformation between two vector spaces V and W is a map $T : V \rightarrow W$ such that the following hold:

- $T(\boldsymbol{v}_1 + \boldsymbol{v}_2) = T(\boldsymbol{v}_1) + T(\boldsymbol{v}_2)$ for any vectors $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$ in V , and
- $T(\alpha \boldsymbol{v}) = \alpha T(\boldsymbol{v})$ for any scalar α .

Modelling Decisions and Implications

$$\hat{y} = w x$$

$$w = -0.47$$



Modelling Decisions and Implications

A linear model:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} := \sum_i w_i x_i$$

From understanding the implications of our mathematical assumption, we can inform the preprocessing decision we make about our algorithm. In this case, normalization:

$$\mathbf{x}^\mu = \frac{1}{n} \sum_{i=1}^n \mathbf{x}^{(i)}, \quad \mathbf{y}^\mu = \frac{1}{n} \sum_{i=1}^n \mathbf{y}^{(i)}$$

Function view

We denote functions with clear domain (input space) and co-domain (output and label space):

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^m$$

We will model our machine learning models as parameterized functions:

$$f^\theta : \mathbb{R}^n \rightarrow \mathbb{R}^m$$

Where we have our linear model, what is the domain and co-domain?

$$f^w(x) := wx$$

Do we need the transpose?

$$f^w(x) := w^\top x$$

Function view

$$\cancel{f^w(x) := wx}$$

$$x \in \mathbb{R}^{n \times 1}$$

$$f^w(x) := w^\top x$$

$$w \in \mathbb{R}^{n \times 1}$$

$wx : (n \times 1)(n \times 1)$ is not defined

$$w^\top x : (1 \times n)(n \times 1) = 1 \times 1, \quad f^w : \mathbb{R}^n \rightarrow \mathbb{R}$$

Function view

$$\cancel{f^w(x) := wx}$$

$$x \in \mathbb{R}^{n \times 1}$$

$$f^w(x) := w^\top x$$

$$w \in \mathbb{R}^{n \times 1}$$

$wx : (n \times 1)(n \times 1)$ is not defined

$$w^\top x : (1 \times n)(n \times 1) = 1 \times 1, \quad f^w : \mathbb{R}^n \rightarrow \mathbb{R}$$

Function view

With the design matrix, we can predict on a whole "batch" of data at once:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}, \quad \mathbf{X} = \begin{bmatrix} \mathbf{x}^{(1)\top} \\ \vdots \\ \mathbf{x}^{(N)\top} \end{bmatrix}$$

$$\mathbf{X} \in \mathbb{R}^{N \times n}, \quad \mathbf{w} \in \mathbb{R}^{n \times 1}$$

$$\mathbf{X}\mathbf{w} : (N \times n)(n \times 1) = N \times 1$$

$$\hat{\mathbf{y}} \in \mathbb{R}^{N \times 1} : \text{one prediction per data point}$$

Function view

$$\hat{y}_i = (\mathbf{X}\mathbf{w})_i$$

$$= \sum_{j=1}^n X_{i,j} w_j$$

row i of $\mathbf{X}$ times $\mathbf{w}$, in summation notation

$$= \sum_{j=1}^n x_j^{(i)} w_j$$

row i of $\mathbf{X}$ is $\mathbf{x}^{(i)\top}$, so $X_{i,j} = x_j^{(i)}$

$$= \mathbf{w}^\top \mathbf{x}^{(i)}$$

a dot product

$$= f^{\mathbf{w}}(\mathbf{x}^{(i)})$$

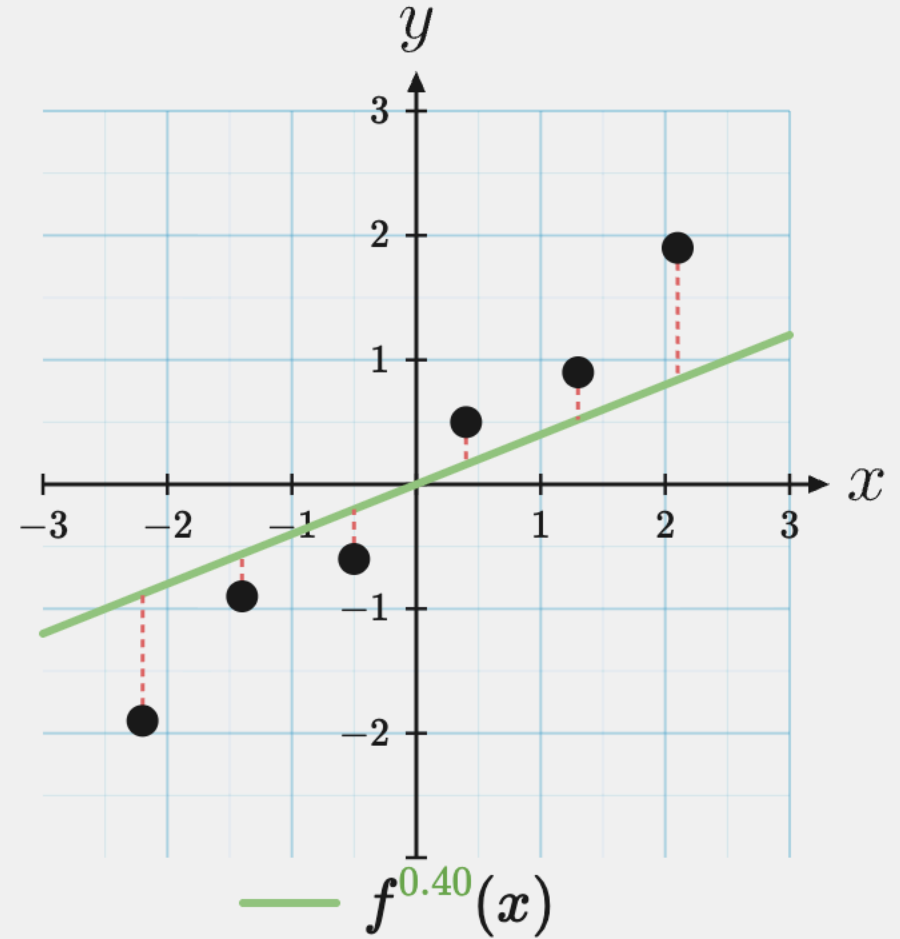
the prediction on the i -th feature vector

Why are the parameter not included in the "domain" of the function?

Function view

The first learning paradigm we will investigate is Empirical Risk Minimization (ERM). It will take several lectures to fully flesh out the foundational ideas, but today we will look at the *minimization* part.

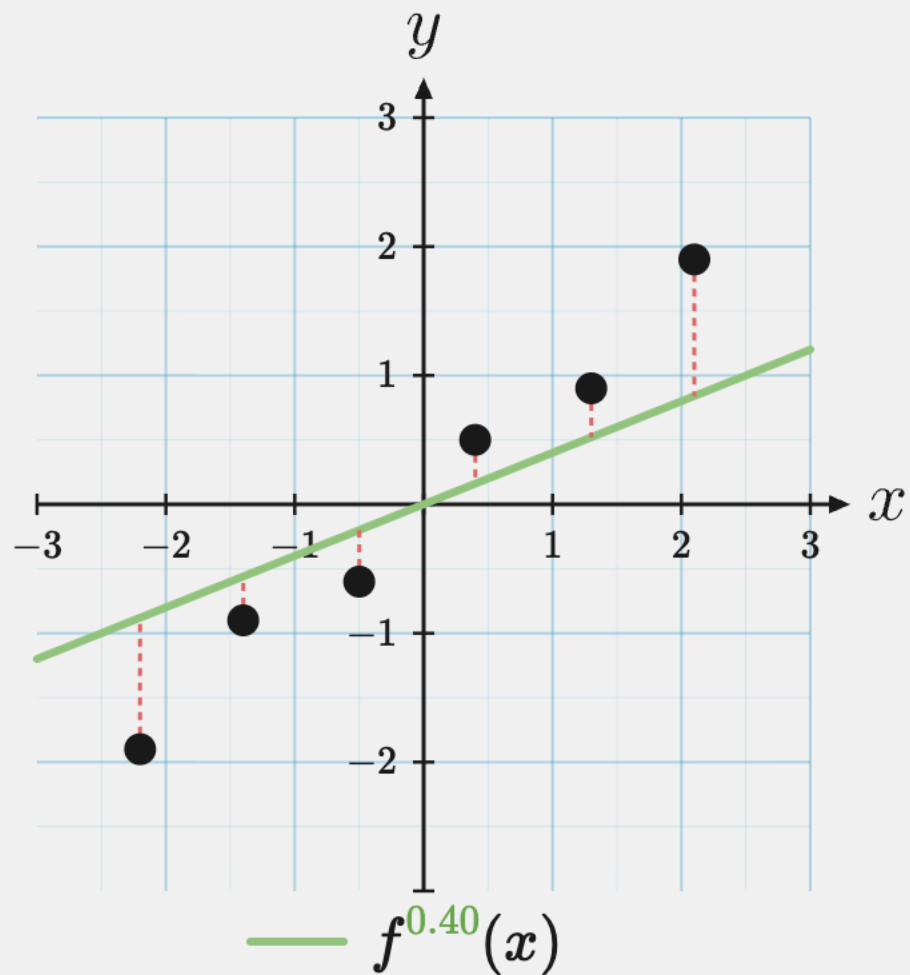
We pose learning as a minimization, namely, what 'w' minimizes our errors?



$$w = 0.40$$



Function view



$$\hat{y} = \begin{bmatrix} -0.88 \\ -0.56 \\ -0.20 \\ 0.16 \\ 0.52 \\ 0.84 \end{bmatrix} \quad y = \begin{bmatrix} -1.9 \\ -0.9 \\ -0.6 \\ 0.5 \\ 0.9 \\ 1.9 \end{bmatrix} \quad \hat{y} - y = \begin{bmatrix} 1.02 \\ 0.34 \\ 0.40 \\ -0.34 \\ -0.38 \\ -1.06 \end{bmatrix}$$

$$w = 0.40$$



Norm Spaces

$$\|\mathbf{x}\|_p = \sqrt[p]{\sum_i |x_i|^p}$$

The p-norm distance, which has some special cases:

$$\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2}$$

The euclidean norm or straight line distance

$$\|\mathbf{x}\|_0 = \sum_i \mathbb{I}(x_i \neq 0)$$

Zero norm or hamming distance

$$\|\mathbf{x}\|_\infty = \sup_i |x_i|$$

Infinity norm or supremum/sup (pronounced like: soup) norm

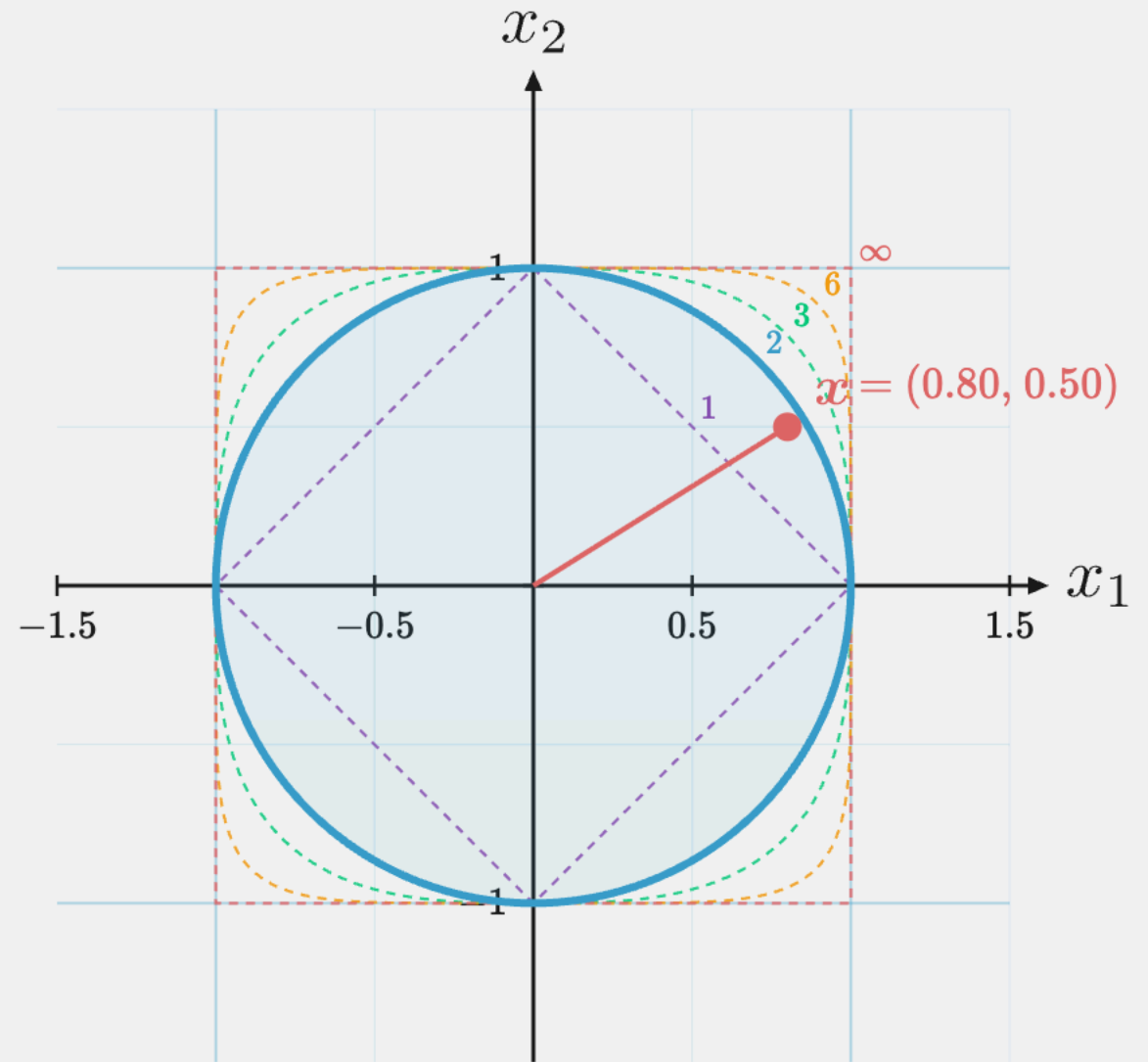
Norm Spaces

$$\|\mathbf{x}\|_p = \sqrt[p]{\sum_i |x_i|^p}$$

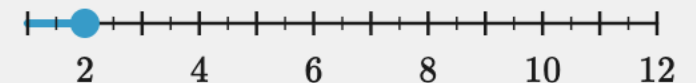
$$\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2}$$

$$\|\mathbf{x}\|_0 = \sum_i \mathbb{I}(x_i \neq 0)$$

$$\|\mathbf{x}\|_\infty = \sup_i |x_i|$$



$p = 2.00$



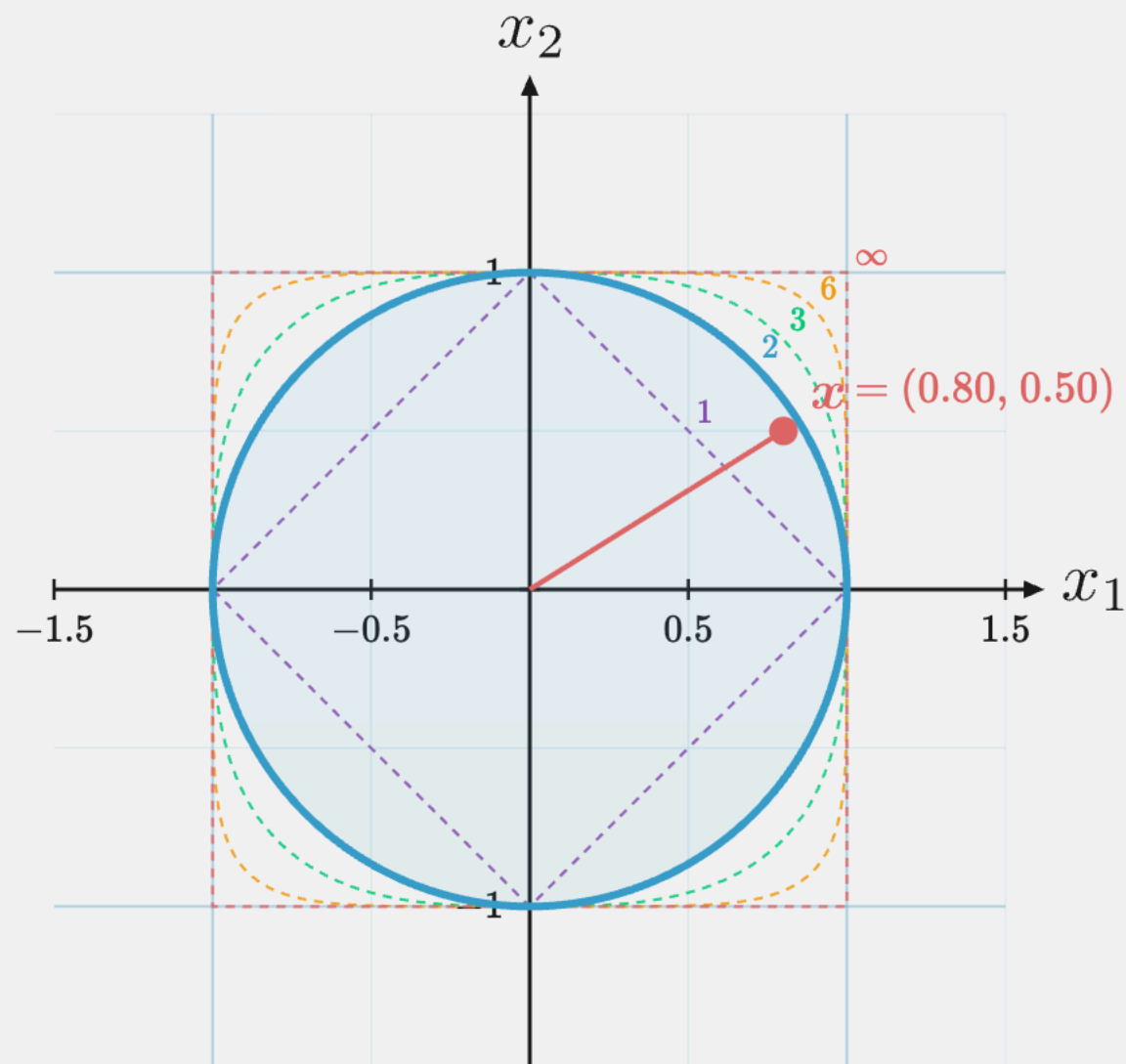
Norm Spaces

$$\|x\|_p = 0.943 \quad (p = 2.00)$$

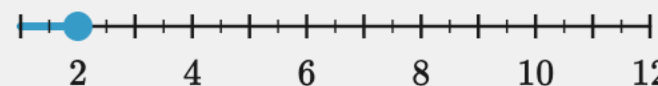
$$\|x\|_2 = 0.943$$

$$\|x\|_0 = 2$$

$$\|x\|_\infty = 0.800$$



$p = 2.00$



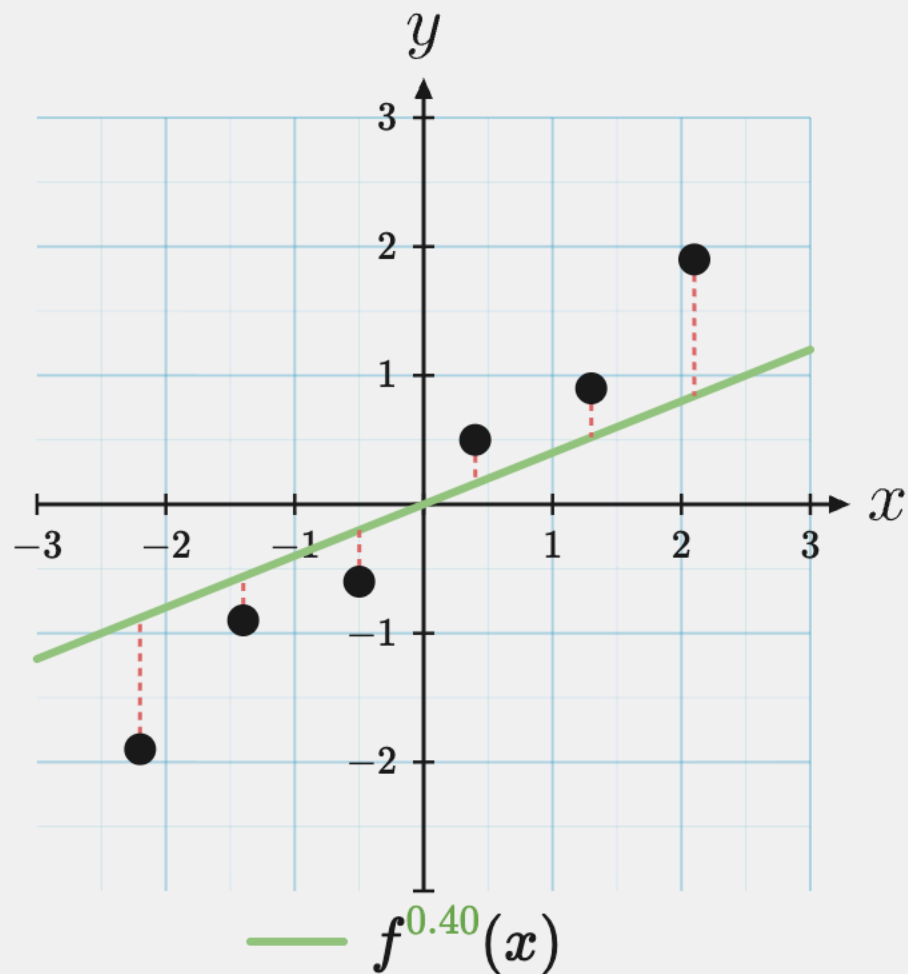
Measuring Error

We use $\mathcal{L}$ as the general notation for the loss function.

Here we take it to be the squared 2-norm of the difference between our predictions and the targets:

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = \|\hat{\mathbf{y}} - \mathbf{y}\|_2^2$$

Measuring Error



$$w = 0.40$$



$$\hat{\mathbf{y}} = \begin{bmatrix} -0.88 \\ -0.56 \\ -0.20 \\ 0.16 \\ 0.52 \\ 0.84 \end{bmatrix}$$

$$\|\hat{\mathbf{y}}\|_p$$

$$1.459$$

$$\mathbf{y} = \begin{bmatrix} -1.9 \\ -0.9 \\ -0.6 \\ 0.5 \\ 0.9 \\ 1.9 \end{bmatrix}$$

$$\|\mathbf{y}\|_p$$

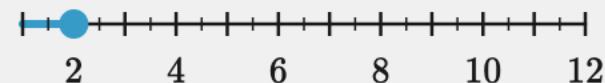
$$3.074$$

$$\hat{\mathbf{y}} - \mathbf{y} = \begin{bmatrix} 1.02 \\ 0.34 \\ 0.40 \\ -0.34 \\ -0.38 \\ -1.06 \end{bmatrix}$$

$$\|\hat{\mathbf{y}} - \mathbf{y}\|_p$$

$$1.643$$

$$p = 2.00$$



Measuring Error

From here on, for the rest of the lecture, we write θ for the model parameters instead of w .

We want to view this error as the error of a parameter, and so we will restate this as follows:

$$\begin{aligned}\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) &= ||\hat{\mathbf{y}} - \mathbf{y}||_2^2 \\ &= ||\mathbf{X}\theta - \mathbf{y}||_2^2 \\ \mathcal{L}(\theta) &= ||\mathbf{X}\theta - \mathbf{y}||_2^2\end{aligned}$$

Norms as “loss” functions

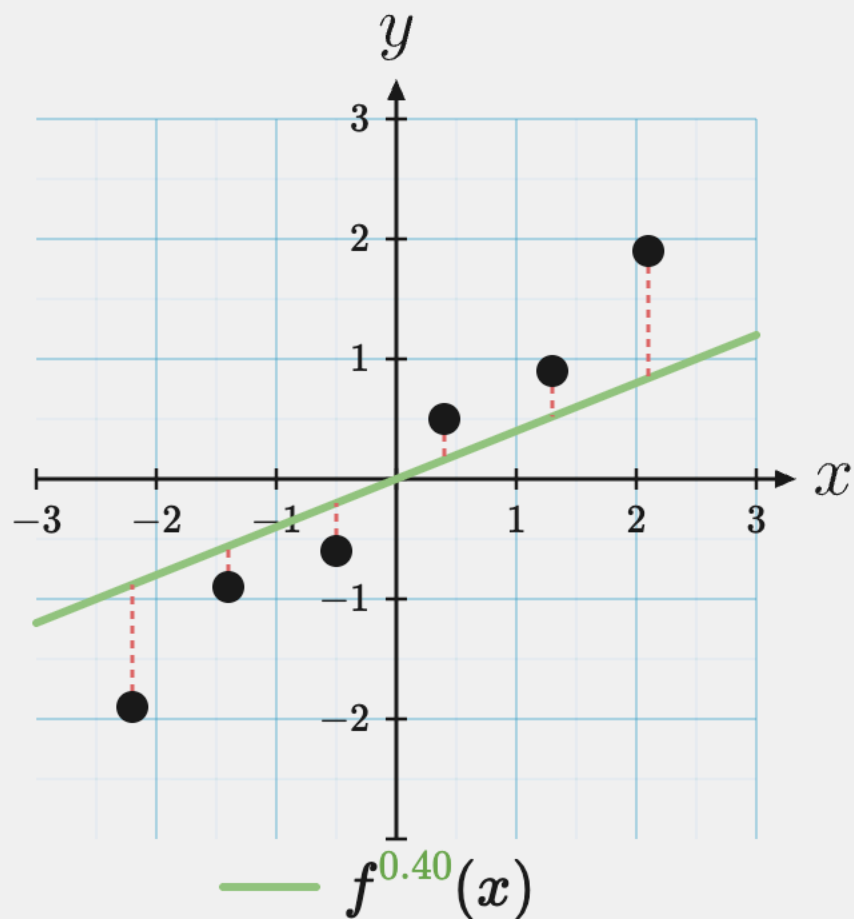
Where we have a vector of desired outputs and a vector of predictions, we can describe the magnitude of their difference using a norm.

$$\mathcal{L}(\theta) = ||\mathbf{X}\theta - \mathbf{y}||_2^2$$

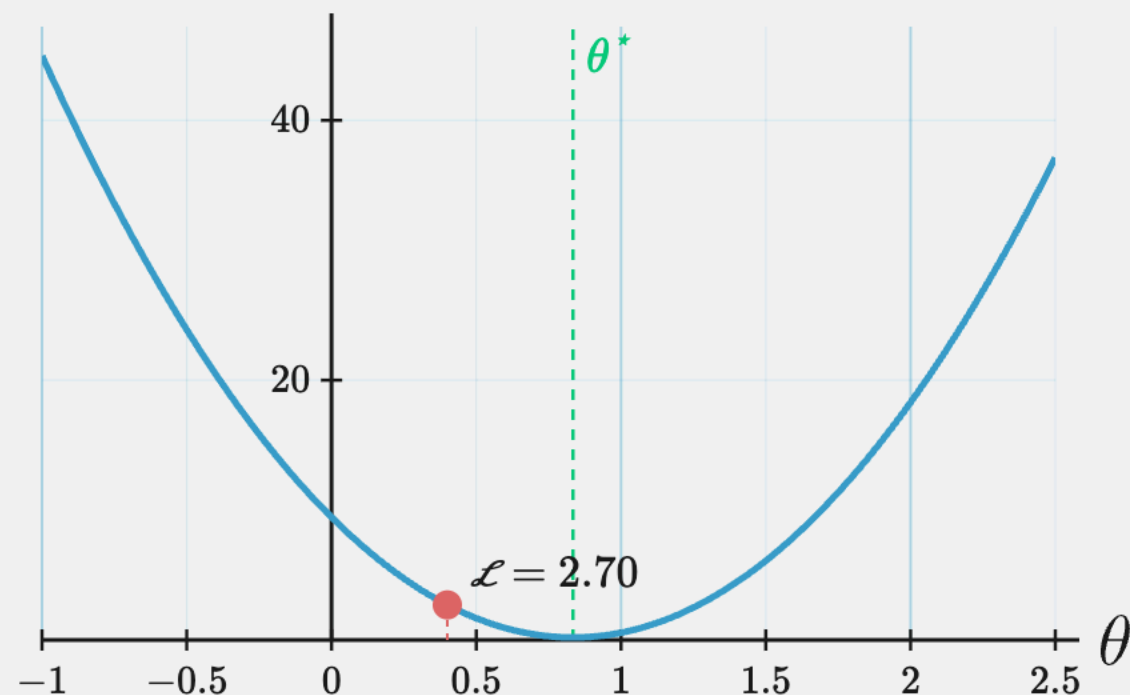
$$\theta^* = \operatorname{argmin}_{\theta \in \mathbb{R}^n} \mathcal{L}(\theta)$$

The 2 norm corresponds to the MSE loss, the 1 norm corresponds to the MAE loss.

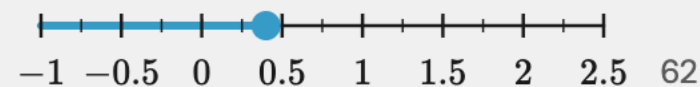
Norms as “loss” functions



$$\mathcal{L}(\theta) = \|X\theta - y\|_2^2$$



$\theta = 0.40$



Norms as “loss” functions

Write out the loss below in summation notation:

$$\mathcal{L}(\theta) = \| \mathbf{X}\theta - \mathbf{y} \|_2^2 = ?$$

Hint: what is the i -th entry of $\mathbf{X}\theta$?

We are ready to state our minimization

Writing the squared 2-norm out entry by entry:

$$\mathcal{L}(\theta) = \|\mathbf{X}\theta - \mathbf{y}\|_2^2 = \sum_{i=1}^m ((\mathbf{X}\theta)_i - y_i)^2$$

the squared 2-norm is the sum of the squared entries

$$(\mathbf{X}\theta)_i = \sum_{j=1}^n X_{i,j} \theta_j$$

entry i of $\mathbf{X}\theta$ is row i of $\mathbf{X}$ times θ

$$\mathcal{L}(\theta) = \sum_{i=1}^m \left(\sum_{j=1}^n X_{i,j} \theta_j - y_i \right)^2 = \sum_{i=1}^m \left(y_i - \sum_{j=1}^n X_{i,j} \theta_j \right)^2$$

We are ready to state our minimization

Now, where we take our loss to be the mean squared error loss (the squared 2-norm) we have the following optimization problem:

$$\mathcal{L}(\theta) = \frac{1}{m} \sum_{i=1}^m \left(y_i - \sum_{j=1}^n X_{i,j} \theta_j \right)^2$$

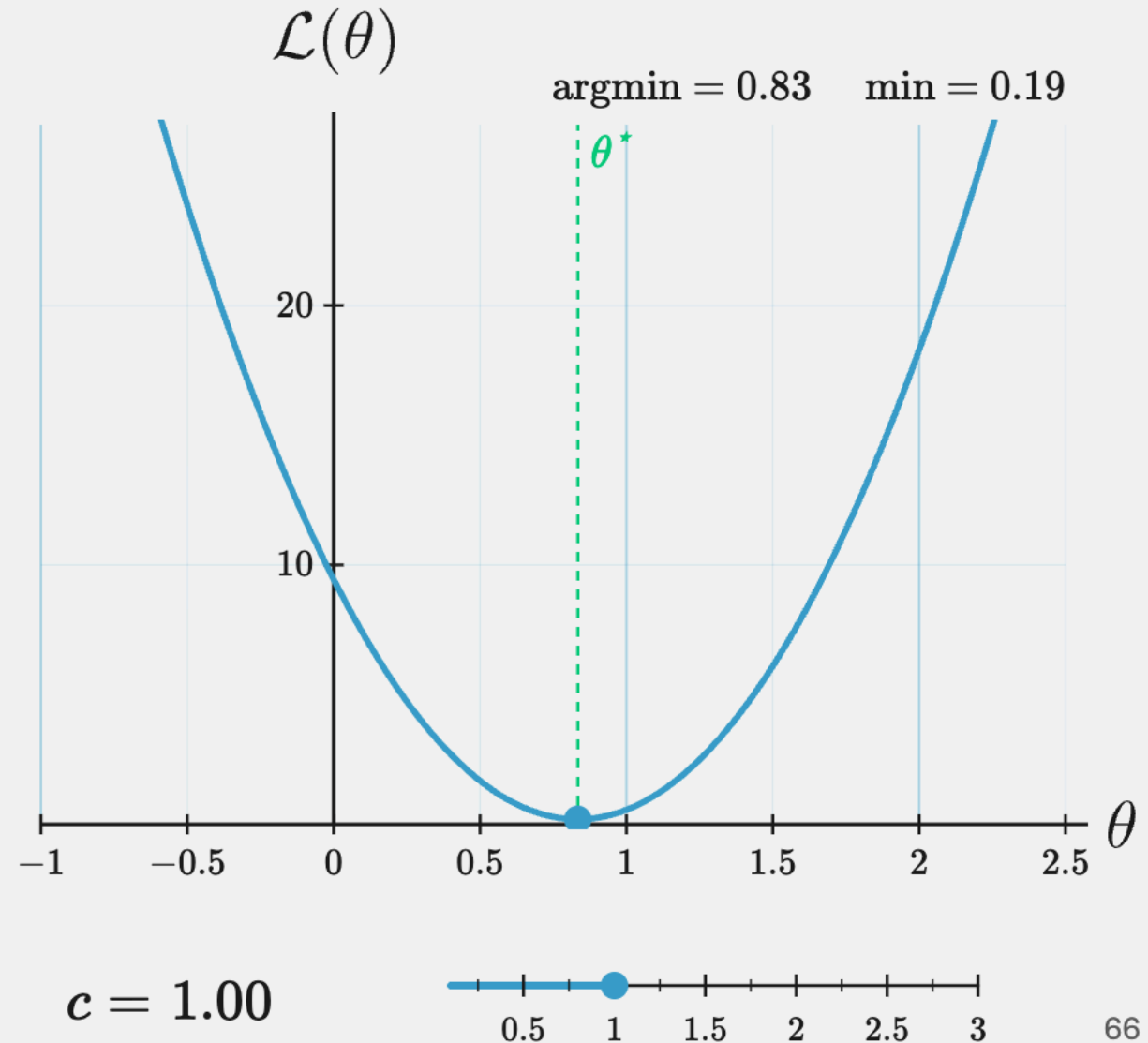
Before this was just a summation, does this 1/m term affect our answer?

We are ready to state our minimization

Before this was just a summation, does this $1/m$ term affect our answer?

No! We care about the argmin.

$$\operatorname{argmin}_{\theta} c \mathcal{L}(\theta) = \operatorname{argmin}_{\theta} \mathcal{L}(\theta) \quad (c > 0)$$



We are ready to state our minimization

Claim: the loss we have just written down can also be written as

$$(\mathbf{X}\boldsymbol{\theta} - \mathbf{y})^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) = \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2$$

Learning Objectives Summary

- Concept of supervised/unsupervised/generative learning
- Definition of classification and regression
- Summation and Index Notation
- Basic data normalization
- P-norm error functions
- Learning as loss function minimization (ERM)

Course Logistics

have given a subjective mark to the difficulty of your practice problems. One common question I get is "how hard is this relative to the exam."

So there are now levels to the practice problems, but this is (1) very subjective and (2) does not mean you will not see a III on your exam.



Feedback



<https://forms.gle/CQUdDJZtV6vJrSeNA>